

Quantum Optimization for the Vehicle Routing Problem (VRP): Qubit-Efficient Encoding Strategies for NISQ Devices

Drestanto Muhammad Dyasputro

Student ID: 1450107

`ddyasputro@student.unimelb.edu.au`

Master of Computer Science

School of Computing and Information Systems

The University of Melbourne

November 23, 2025

Supervisor: Prof Udaya Parampalli

`(udaya@unimelb.edu.au)`

Abstract

The Vehicle Routing Problem (VRP) is a fundamental combinatorial optimization problem with broad applications in logistics, supply chain management, and transportation planning. As an NP-hard problem, VRP exhibits exponential computational complexity that renders exact classical algorithms impractical for large-scale instances. This motivates the exploration of quantum computing approaches that can provide good approximate solutions. This thesis investigates a qubit-efficient encoding strategy for solving VRP using the Quantum Approximate Optimization Algorithm (QAOA) on Noisy Intermediate-Scale Quantum (NISQ) devices. The proposed node-visit-time encoding reduces qubit requirements from N^2 (naive adjacency encoding) to $N \cdot \lceil \log_2(N) \rceil$. Through systematic experiments on IBM Quantum Cloud across 23 problem instances, we demonstrate that node-visit-time encoding achieves a 91.3% success rate compared to naive adjacency's 43.5%. While naive adjacency achieves 98.7% optimality ratio when successful, node-visit-time maintains competitive solution quality with 92.8% average optimality ratio. This novel qubit-efficient encoding strategy enables practical quantum optimization of VRP on current NISQ hardware, providing a foundation for scaling to larger problem instances as quantum devices continue to improve.

Declaration

This is to certify that:

1. the thesis comprises only my original work towards the Master of Computer Science except where indicated in the text,
2. due acknowledgement has been made in the text to all other material used,

Drestanto Muhammad Dyasputro

Date: November 23, 2025

Acknowledgements

I would like to express my sincere gratitude to my supervisors, Prof Udaya Parampalli and Shashank Bhat for their invaluable guidance, support, and encouragement throughout this research project. Their expertise in quantum computing and quantum optimization has enabled me to finish this work.

I am grateful to the University of Melbourne for providing the resources and academic environment necessary for conducting this research. I also thank IBM Quantum for providing access to their quantum computing platform, which made the experimental validation of this work possible.

I also acknowledge LPDP Endowment Fund for giving me a scholarship to study and research in the University of Melbourne.

Finally, I extend my heartfelt thanks to my family and friends for their unwavering support and patience during the completion of this thesis.

Contents

Abstract	1
Declaration	2
Acknowledgements	3
List of Figures	8
List of Tables	9
1 Introduction	10
1.1 The Vehicle Routing Problem Explained	11
1.2 Case Study: Waste Collection in the City of Casey	11
1.3 Solving the VRP	11
1.3.1 Quantum Programming Approaches for VRP	12
1.4 Problem Statement	12
1.5 Research Steps	13
2 Background Theory and Literature Review	13
2.1 The Vehicle Routing Problem Explained: Variants and Applications . . .	13
2.2 Classical Approaches to VRP	15
2.2.1 Exact Algorithms	15
2.2.2 Metaheuristic Approaches	16
2.3 Quantum Computing Fundamentals	16
2.3.1 Qubits and Quantum States	17
2.3.2 Quantum Gates and Circuits	17
2.3.3 Measurement and the Measurement Problem	17
2.3.4 NISQ Devices	17
2.4 Quantum Approaches to Combinatorial Optimization	17
2.4.1 Quantum Approximate Optimization Algorithm (QAOA)	18
2.5 Quantum Walk-based Approaches	18
2.6 Quantum Annealing for VRP	19
2.7 Hybrid Quantum-Classical Methods and Machine Learning	20
2.7.1 Variational Quantum Algorithms (VQAs)	20
2.7.2 Quantum Support Vector Machines for VRP	20
2.7.3 Quantum-Inspired Classical Algorithms	21
2.8 Qubit-Efficient Encoding and Allocation Research	21
2.9 Other Important Research on Quantum Optimization	22
2.9.1 Quantum Algorithms on TSP and VRP	22

2.9.2	Quantum Circuit Optimization	22
2.10	Summary of Literature Review	22
3	Methodology and Research Design	23
3.1	Research Approach	23
3.2	Case Study: Waste Collection in the City of Casey	23
3.3	VRP Model Construction	24
3.3.1	Problem Instance Generation	25
3.3.2	Constraint Handling	25
3.4	QAOA Implementation for VRP	26
3.4.1	VRP Representation in Quantum Qubits	26
3.4.2	Qubit Allocation Improvements and Discussion	26
3.4.3	Cost Function for Naive Matrix-Based Encoding	28
3.4.4	Cost Function for Logarithmic Permutation Encoding	29
3.4.5	Cost Function for Node-Visit-Time Encoding	29
3.4.6	Qubit Allocation for Depots and Vehicles	31
3.4.7	Quantum Circuit and Hamiltonian	33
3.4.8	Addressing Constraints	34
3.5	Other Notes	35
3.5.1	Memory Problem	35
3.5.2	Comparison of Operation Number (Classical vs. Quantum)	35
3.5.3	Using v Vehicles and d Depots	35
4	Experimental Directions and Setup	36
4.1	Experimental Directions and Some Justification	36
4.1.1	Comparison of VRP Solvers: Naive vs. Google OR-Tools	36
4.1.2	Research Directions on These Experiments	38
4.1.3	Optimizers Justification in QAOA for the VRP	40
4.2	Experimental Setup	42
4.2.1	Data Preprocessing)	42
4.2.2	VRP Instance Details	44
4.2.3	Node-Visit-Time Encoding	44
4.2.4	Cost Function Implementation	45
4.2.5	Conceptual Link to VRP Cost Function (Implicit Hamiltonian) .	45
4.2.6	QAOA Circuit Construction (<code>create_qaoa_circuit</code>)	46
4.2.7	Objective Function for Classical Optimizer	48
4.2.8	Performance Metrics and Data Analysis Plan	48
5	Results	51
5.1	Summary of Key Results	51

5.2	Optimality Ratio	52
5.3	Invalid Bitstring Measurement	54
5.4	Comparison of Encoding Techniques	56
5.5	Impact of Number of Shots	57
5.6	QAOA Depth Analysis	58
5.7	Cost Distribution Analysis	60
5.8	Scaling Analysis	61
5.9	Limitations and Future Work	62
5.10	Overall Performance Summary	62
6	Discussions	63
6.1	Why Node-Visit-Time Encoding Outperforms Naive Adjacency	63
6.1.1	Constraint Structure and Hamiltonian Locality	63
6.1.2	State Space Structure	63
6.1.3	Gradient Landscape and Parameter Optimization	64
6.1.4	Error Resilience	64
6.2	The Theory-Practice Gap in Qubit Efficiency	65
6.2.1	Why Logarithmic Encoding Fails in Practice	65
6.3	Implications of High Invalid Bitstring Rates	65
6.3.1	Sources of Invalid Bitstrings	66
6.3.2	Mitigation Strategies	66
6.4	QAOA Parameter Optimization: Depth vs. Shots Trade-Off	67
6.4.1	The Depth Plateau	67
6.4.2	The Shot Count Plateau	68
6.5	Quantum Advantage Analysis	68
6.5.1	Current State: No Quantum Advantage	68
6.5.2	Theoretical Scaling Arguments	69
6.6	Scalability: Current Limitations and Future Prospects	69
6.6.1	Physical Limitations of Current NISQ Devices	69
6.6.2	Algorithmic Approaches to Improved Scaling	70
6.6.3	Hardware Roadmap and Quantum Advantage Timeline	70
6.7	Comparison with Related Work	71
6.7.1	Consistency with Prior QAOA Studies	71
6.7.2	Comparison with Quantum Annealing Approaches	72
6.7.3	Novel Contributions of This Work	72
6.8	Practical Recommendations for Quantum VRP Research	73
6.8.1	Encoding Strategy Selection	73
6.8.2	QAOA Parameter Selection	73
6.8.3	Hardware and Implementation	73

6.8.4	Research Directions	74
7	Conclusion	74
7.1	Summary of Key Findings	74
7.1.1	Encoding Strategy Performance	74
7.1.2	QAOA Parameter Optimization	75
7.1.3	NISQ Limitations and Scalability	75
7.1.4	Quantum Circuit Behavior	75
7.1.5	Classical vs. Quantum Performance	75
7.2	Research Contributions	76
7.3	Limitations and Challenges	76
7.3.1	Hardware Specificity	76
7.3.2	Problem Instance Scope	77
7.3.3	Classical Baseline Selection	77
7.3.4	Algorithm Scope	77
7.3.5	Error Mitigation Absence	77
7.4	Future Research Directions	77
7.4.1	Algorithmic Innovations	77
7.4.2	Error Mitigation and Hardware Optimization	78
7.4.3	Algorithmic Comparisons	78
7.4.4	Application Extensions	79
7.4.5	Theoretical Investigations	79
7.5	Broader Impact and Significance	79
7.6	Closing Remarks	80
A	Software Implementation and Repository	82
A.1	Repository Access	82
A.2	Repository Structure	82
A.3	Running the Experiments	82

List of Figures

1	Illustration of VRP solution generation: (left) unassigned customers, (middle) customer clustering, (right) optimized routes [1].	15
2	QAOA circuit structure with more than one depth ($p > 1$): Hadamard initialization, alternating cost and mixer layers.	34
3	Invalid bitstring counts for naive adjacency vs. node-visit-time encodings across VRP instances.	56
4	Optimality ratio trend across various measurement shot counts (ranging from 128 to 4096) for node-visit-time encoding.	58
5	Optimality ratio trend across various QAOA circuit depths (ranging from $p = 2$ until $p = 6$) for node-visit-time encoding.	59
6	Normalized cost distribution (KDE plotted) for node-visit-time encoding. 0 means optimal and 1 means worst.	60
7	Optimality ratio across various number of nodes (ranging from 3 to 5 nodes) for two encoding strategies.	61

List of Tables

1	Comparison of two qubit encoding strategies for QAOA across 23 VRP problem instances through several metrics.	52
2	Optimality ratio vs. measurement shots for node-visit-time encoding. . .	57
3	Optimality ratio and actual cost compared across various QAOA depth for node-visit-time encoding.	59
4	Optimality ratio and success rate compared across various problem size for two encoding strategies.	61

1 Introduction

The vehicle Routing Problem (VRP) is a fundamental problem in logistics studied for finding optimum delivery routes for several vehicles in order to visit a series of customers. Over the years, VRP has been studied extensively, as the problem can be defined in many versions with various assumptions and constraints. As a consequence, VRP has a broad range of variants that researchers must consider.

The VRP is classified as NP-hard [2] because its computational complexity grows exponentially with the size of the input. As a consequence, exact classical algorithms quickly become impractical for large-scale instances. Due to this NP-hard nature, approximate methods such as heuristics and metaheuristics become essential for finding near-optimal solutions in reasonable time. This characteristic also justifies exploring quantum optimization approaches, which may offer computational advantages for such combinatorial problems. In practice, the problem becomes even more challenging when real-world constraints are incorporated, such as traffic conditions, weather variations, and road incidents.

To construct a realistic case study, this research integrates these practical considerations into a VRP instance using datasets provided by the City of Casey. The core operational data include information on depots [3], customers [4, 5], and the underlying road network or routes [6]. These are complemented with contextual datasets describing traffic conditions [7], weather patterns [8], and road incidents or crashes [9]. Collectively, these sources offer a comprehensive representation of the operational environment relevant to solving VRP instances.

Quantum computing represents a computational paradigm that leverages quantum mechanical phenomena to process information. Unlike classical bits that exist in either a 0 or 1 state, quantum bits (qubits) can exist in a *superposition* of both states simultaneously, enabling the exploration of multiple solution paths in parallel. Furthermore, qubits can exhibit *entanglement*, a quantum correlation that allows the state of one qubit to depend on the state of another, regardless of distance between them. These properties enable quantum computers to approach certain computational problems in ways that are fundamentally different from classical computers.

Building on these quantum mechanical principles, quantum computing offers a promising paradigm for approaching and solving combinatorial optimization problems such as VRP. The Quantum Approximate Optimization Algorithm (QAOA)[10], introduced by Farhi et al. in 2014, is specifically designed to find approximate solutions to combinatorial optimization problems by alternating between problem-specific and mixing Hamiltonians to explore the solution space. Other quantum approaches, such as Quantum Walk[11]-

based Optimization, have also shown promise in tackling NP-hard tasks. This research proposes to explore QAOA to develop approximation methods for solving the VRP.

1.1 The Vehicle Routing Problem Explained

VRP has numerous real-world applications, including logistics, supply chain management, and last-mile delivery services. Various VRP variants exist, such as the Capacitated VRP (CVRP), Time-Window VRP (VRPTW), and Stochastic VRP (SVRP), each introducing unique constraints that impact solution complexity. Classical methods struggle to find optimal solutions as problem size increases due to exponential computational growth. A detailed illustration of VRP solution methodology is provided in Section 2.1 (see Figure 1).

In this study, I focus on a general VRP formulation without vehicle capacity constraints (as in Capacitated VRP or CVRP) or time window restrictions (as in Time-Window VRP or VRPTW). Constraints used are limited to the three types discussed in the Section 7.3.2.

1.2 Case Study: Waste Collection in the City of Casey

To ground this research in practical applications, I utilize real-world data from the City of Casey waste collection system in Victoria, Australia. This case study provides realistic VRP instances with publicly accessible datasets while maintaining problem scales suitable for current NISQ hardware. The detailed methodology for constructing the VRP model from this data is presented in Section 3.

1.3 Solving the VRP

VRP can be solved using classical algorithm (without quantum algorithm). Classical methods for solving the VRP include:

- Combinatorial Naive Method: Trying all possible combinations to get the minimum path.
- Combinatorial optimized methods like the branch-and-bound for CVRP[12].
- Heuristic Techniques such as Genetic Algorithm[13] and Ant Colony Optimization[13] that offer usable sub-optimal solutions.

Exact algorithms can be used to solve VRP and get the true optimal solution, while heuristic techniques offer a near-optimal solution.

Quantum computing offers a novel approach to solving VRP by leveraging quantum superposition and entanglement to explore solutions. Quantum algorithms like QAOA[10] Quantum Walk[11] methods have shown potential in improving combinatorial optimiza-

tion efficiency. However, implementing these algorithms for practical VRP scenarios requires addressing efficiency, scalability, and quantum hardware limitations.

1.3.1 Quantum Programming Approaches for VRP

Quantum Optimization Techniques can generate better solutions compared to the classical optimization techniques. But more importantly, faster than the exact classical algorithms. Several approaches have been proposed to explore the advantages of quantum algorithms over classical optimization methods, particularly for NP-hard problems like the Vehicle Routing Problem (VRP):

- Quantum Approximate Optimization Algorithm (QAOA): Designed to find approximate solutions to combinatorial problems[10] by leveraging quantum superposition and entanglement.
- Quantum Walk-based Optimization: Quantum walks, the quantum analog of classical random walks, have been applied to search and optimization problems[14], offering potential speedups over classical methods.
- Variational Quantum Eigensolver[15] (VQE): Studied for solving VRP, though currently limited to small instances.
- Quantum Annealing[16]: A novel QUBO formulation[16] has been proposed for CVRP, incorporating a time-table representation to model the time evolution of each vehicle.

1.4 Problem Statement

The central challenge addressed in this thesis is: **How can we efficiently encode Vehicle Routing Problem instances into quantum circuits to enable practical quantum optimization?**

More specifically, this research investigates the following critical questions:

1. What are the fundamental trade-offs between different qubit allocation strategies for encoding VRP in quantum circuits, and how do various encoding approaches perform in practice?
2. How does the performance of quantum approaches compare with classical benchmarks in terms of solution quality and success rates?

While theoretical computer science can most of the time (not all of the time) established lower bounds on the number of qubits required to represent combinatorial problems, practical implementation reveals that theoretical optimality does not necessarily translate to practical effectiveness.

1.5 Research Steps

The primary steps of this research (including the objective of each step) are:

1. **Theoretical Analysis:** Develop a comprehensive theoretical framework for understanding qubit allocation strategies for VRP, analyzing the trade-offs between qubit count, circuit complexity, and problem representation fidelity.
Output: Research Methodology
2. **Algorithm Development:** Implement and optimize QAOA-based VRP solvers using multiple encoding strategies, including naive adjacency matrix encoding (N^2 qubits), logarithmic permutation encoding ($\lceil \log_2(N!) \rceil$ qubits), and node-visit-time encoding ($N \cdot \lceil \log_2(N) \rceil$ qubits).
Output: Research Implementation Design
3. **Experimental Validation:** Conduct systematic experiments on IBM Quantum Cloud to evaluate the performance of different encoding strategies across multiple problem instances, analyzing success rates, solution quality, and invalid bitstring percentages.
Output: Research Output Reports and Insights
4. **Comparative Benchmarking:** Compare quantum optimization results against classical optimal solutions to quantify the solution quality (optimality ratio) achieved by quantum approaches.
Output: Research Conclusion

2 Background Theory and Literature Review

This section establishes the theoretical foundation for the study and provides a literature review that guides the research direction. We begin by explaining the Vehicle Routing Problem and reviewing key classical solution approaches, followed by an introduction to essential quantum computing concepts. For each topic, we first outline the fundamental ideas and definitions, then expand into a detailed literature review highlighting prior work. Finally, we examine existing quantum approaches to combinatorial optimization and identify the gaps that this research aims to address.

2.1 The Vehicle Routing Problem Explained: Variants and Applications

The Vehicle Routing Problem (VRP) encompasses a broad family of optimization problems with extensive real-world applications spanning logistics, supply chain management,

last-mile delivery services, waste collection, emergency response, and many other domains. Since its inception, VRP has evolved into numerous variants, each introducing specific constraints and objectives that reflect real-world operational requirements[2].

Common VRP Variants:

- **Capacitated VRP (CVRP):** The most fundamental variant where vehicles have limited carrying capacity. Each customer has a demand that must be satisfied, and no vehicle can exceed its capacity constraint. This variant directly applies to distribution centers delivering goods to retail locations.
- **VRP with Time Windows (VRPTW):** Extends CVRP by adding temporal constraints where each customer must be served within a specified time window. This variant is crucial for applications like meal delivery services or scheduled waste collection.
- **VRP with Multiple Depots (MDVRP):** Involves multiple depot locations from which vehicles can start and end their routes. This reflects the reality of organizations with distributed facilities.
- **VRP with Pickup and Delivery (VRPPD):** Vehicles must both pick up and deliver goods, often with precedence constraints. This applies to courier services and ride-sharing applications.
- **Stochastic VRP (SVRP):** Incorporates uncertainty in parameters such as customer demands, travel times, or customer presence. This variant addresses real-world variability more realistically.
- **Dynamic VRP (DVRP):** Addresses scenarios where customer requests arrive in real-time and routes must be updated dynamically. This is essential for on-demand services like Uber or food delivery platforms.

Classical approaches to VRP solution methods can be categorized into two main classes: exact algorithms and approximation methods. As problem size increases, the exponential computational growth makes exact solutions impractical, necessitating heuristic and metaheuristic approaches[12].

Figure 1 illustrates a typical VRP solution methodology. The left panel shows unassigned customers (numbered circles) scattered around a central depot (labeled “D”). The middle panel demonstrates customer clustering, where customers are grouped into different sets based on locations, demands, or distances. This is where heuristics come into play, with each color representing a specific vehicle’s assigned customer group. The right panel shows the final optimized routes, where each vehicle follows an efficient path starting and ending at the depot, minimizing total travel distance and cost.

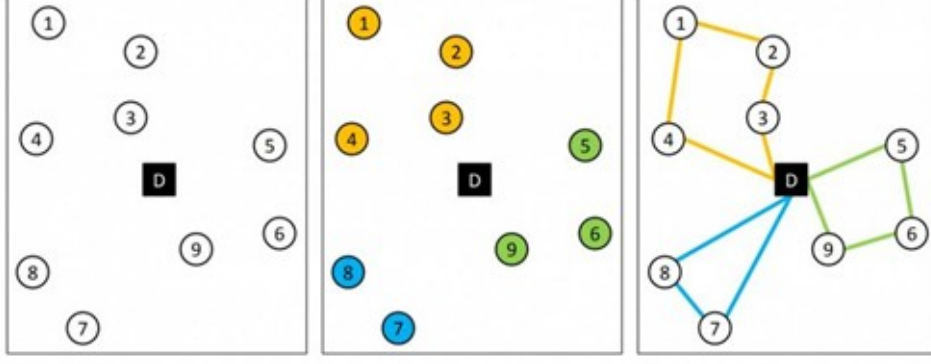


Figure 1: Illustration of VRP solution generation: (left) unassigned customers, (middle) customer clustering, (right) optimized routes [1].

2.2 Classical Approaches to VRP

Classical methods for solving VRP have been extensively developed over the past six decades. These can be broadly categorized into exact algorithms and approximation methods[13].

2.2.1 Exact Algorithms

Exact algorithms guarantee finding the optimal solution but face severe computational limitations as problem size grows:

- **Branch-and-Bound:** Systematically explores the solution space by branching on decision variables and bounding subproblems to prune unpromising branches. While theoretically complete, branch-and-bound becomes impractical for instances with more than 50–100 customers due to exponential tree growth.
- **Branch-and-Cut:** Enhances branch-and-bound by dynamically generating cutting planes to tighten linear programming relaxations. This approach has successfully solved CVRP instances with up to several hundred customers but requires sophisticated implementation and significant computational resources[12].
- **Dynamic Programming:** Decomposes the problem into overlapping subproblems. The Held-Karp algorithm for TSP (a special case of VRP) achieves $O(n^2 2^n)$ complexity, still exponential but better than the $O(n!)$ brute-force approach.
- **Integer Linear Programming (ILP):** Formulates VRP as an integer programming problem and employs commercial solvers like CPLEX or Gurobi. While powerful for small to medium instances, ILP solvers struggle with large-scale problems due to the exponential number of feasible solutions.

2.2.2 Metaheuristic Approaches

Given the computational intractability of exact methods for large instances, metaheuristics have become the practical choice for industrial applications[13]:

- **Genetic Algorithms (GA):** Employ evolutionary principles including selection, crossover, and mutation to evolve populations of solutions toward optimality. GAs have shown success in VRP but require careful parameter tuning and can be computationally expensive.
- **Simulated Annealing (SA):** Mimics the physical annealing process by probabilistically accepting worse solutions to escape local optima. SA provides a good balance between solution quality and computational efficiency.
- **Ant Colony Optimization (ACO):** Inspired by ant foraging behavior, ACO constructs solutions through pheromone-guided probabilistic decisions. ACO has proven particularly effective for VRP variants[13].
- **Tabu Search:** Maintains a memory structure (tabu list) to prevent cycling and guide the search toward unexplored regions. Tabu search has achieved state-of-the-art results for many VRP benchmarks.
- **Variable Neighborhood Search (VNS):** Systematically changes neighborhood structures during local search to escape local optima. VNS offers simplicity with competitive performance.
- **Large Neighborhood Search (LNS):** Iteratively destroys and repairs large portions of the current solution. This approach has shown excellent results especially for time-window constrained VRP.

Despite these advances, classical algorithms face fundamental limitations imposed by the NP-hard nature of VRP. As problem instances approach real-world scales (hundreds or thousands of customers), even the best metaheuristics require prohibitive computational time to guarantee near-optimal solutions. This computational barrier motivates exploring alternative computational paradigms, particularly quantum computing.

2.3 Quantum Computing Fundamentals

Before examining quantum approaches to VRP, we briefly introduce key quantum computing concepts relevant to optimization algorithms.

2.3.1 Qubits and Quantum States

Unlike classical bits that exist in states 0 or 1, quantum bits (qubits) can exist in superposition: a linear combination of basis states $|0\rangle$ and $|1\rangle$:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle$$

where $\alpha, \beta \in \mathbb{C}$ and $|\alpha|^2 + |\beta|^2 = 1$. For an n -qubit system, the quantum state spans a 2^n -dimensional Hilbert space, enabling exponential state representation.

2.3.2 Quantum Gates and Circuits

Quantum computation proceeds through unitary operations (quantum gates) applied to qubits. Common single-qubit gates include Pauli matrices (X, Y, Z), Hadamard (H), and rotation gates (R_x, R_y, R_z). Multi-qubit gates like CNOT enable entanglement, a key quantum resource. A quantum circuit comprises a sequence of gates applied to an initial state, typically $|0\rangle^{\otimes n}$.

2.3.3 Measurement and the Measurement Problem

Measuring a quantum state collapses it to a classical bitstring according to the Born rule, with probability $|\alpha_i|^2$ for obtaining outcome i . This probabilistic nature necessitates multiple measurements (shots) to estimate the quantum state's probability distribution.

2.3.4 NISQ Devices

Current quantum computers operate in the Noisy Intermediate-Scale Quantum (NISQ) era, characterized by:

- Limited qubit counts (tens to hundreds)
- Short coherence times (microseconds to milliseconds)
- Imperfect gate operations with error rates $\sim 10^{-3}$ to 10^{-2}
- Limited or no error correction

These constraints profoundly impact algorithm design, favoring shallow circuits and hybrid quantum-classical approaches.

2.4 Quantum Approaches to Combinatorial Optimization

Several quantum algorithmic frameworks target combinatorial optimization problems like VRP:

2.4.1 Quantum Approximate Optimization Algorithm (QAOA)

QAOA, introduced by Farhi, Goldstone, and Gutmann in 2014[10], represents a hybrid quantum-classical algorithm designed specifically for combinatorial optimization on NISQ devices. QAOA encodes an optimization problem’s cost function into a quantum Hamiltonian and constructs a parameterized quantum circuit to explore the solution space. QAOA is useful for VRP because it can efficiently find approximate solutions to complex optimization problems, including route planning. VRP involves selecting the best paths for multiple vehicles while considering constraints like distance, capacity, and time windows. Since VRP is an NP-hard problem, classical algorithms struggle to solve large instances efficiently. QAOA leverages quantum superposition and interference to explore multiple possible solutions simultaneously, potentially finding good routes faster than classical heuristics.

Mathematically, QAOA encodes the VRP as a cost function H_C , which represents the total travel distance or other constraints, and a mixing function H_M , which helps the algorithm explore different routes. The quantum circuit is defined as:

$$|\psi(\gamma, \beta)\rangle = U_M(\beta_p)U_C(\gamma_p) \cdots U_M(\beta_1)U_C(\gamma_1)|s\rangle$$

where $|s\rangle$ is the initial uniform superposition state (typically prepared by applying Hadamard gates to $|0\rangle^{\otimes n}$), $U_C(\gamma) = e^{-i\gamma H_C}$ encodes the VRP constraints through the cost Hamiltonian with angle parameter γ , and $U_M(\beta) = e^{-i\beta H_M}$ applies the mixing Hamiltonian with angle parameter β to allow transitions between different solutions. The parameters $\{\gamma_1, \dots, \gamma_p, \beta_1, \dots, \beta_p\}$ are optimized classically to minimize the expectation value $\langle\psi(\gamma, \beta)|H_C|\psi(\gamma, \beta)\rangle$. Fitzek et al.[17] applied this method to handle vehicle-specific constraints, improving efficiency. Leonidas et al.[18] optimized QAOA for small quantum devices by reducing qubit requirements. Farhi et al.[10] originally introduced QAOA, showing that deeper circuits can improve results, though real quantum hardware still has limitations.

While QAOA represents the dominant gate-based approach, alternative quantum paradigms offer complementary perspectives on combinatorial optimization.

2.5 Quantum Walk-based Approaches

Quantum walks, which serve as a quantum analog to classical random walks, have been explored for VRP optimization. Quantum walks are useful for VRP because they can explore large search spaces efficiently by leveraging quantum interference and superposition. Unlike classical random walks, where the probability of moving to a new state depends only on the previous step, quantum walks maintain phase coherence, allowing for faster

propagation through the solution space. The evolution of a discrete-time quantum walk is governed by the unitary operator:

$$U = S \cdot (I \otimes C)$$

where S is the shift operator that moves the walker between states, I is the identity operator on the position space, C is the coin operator that controls the direction of movement, and $\otimes$ denotes the tensor product. Bennett et al.[14] applied this framework to vehicle routing, demonstrating that quantum walks can help find optimal paths by efficiently traversing possible routes. Marsh and Wang[11] introduced an approximation method based on quantum walks, showing its applicability to bounded NP-hard problems. In a later study, they improved the efficiency of combinatorial optimization by refining the quantum walk formulation to enhance solution quality[19]. These works suggest that quantum walks could provide speedups for VRP compared to classical heuristics, though practical implementation remains limited by current quantum hardware constraints.

Beyond gate-based quantum computing, specialized quantum hardware offers alternative approaches to optimization problems.

2.6 Quantum Annealing for VRP

Quantum annealing has been another area of research in VRP optimization. Tambunan et al. (2023) investigated quantum annealing for VRP with weighted segments, focusing on integrating dynamic constraints within the optimization process. Similarly, Irie et al. (2019) formulated a Quadratic Unconstrained Binary Optimization (QUBO) model for VRP with time, state, and capacity constraints, showcasing quantum annealing’s applicability in complex routing scenarios.

Quantum annealing (QA) is a heuristic algorithm used for solving optimization problems. It relies on the principle of quantum mechanics, specifically the quantum superposition and tunneling phenomena, to find the global minimum of a cost function. The general form of a Hamiltonian used in quantum annealing for VRP can be expressed as:

$$H(t) = A(t)H_P + B(t)H_D$$

where t represents time during the annealing process, H_P is the problem Hamiltonian that encodes the VRP objective and constraints, H_D is the driver Hamiltonian that governs the evolution of the quantum state, and the time-dependent functions $A(t)$ and $B(t)$ control the mixing between these Hamiltonians during the annealing process (typically with $A(0) = 0$, $A(T) = 1$, $B(0) = 1$, $B(T) = 0$ where T is the total annealing time). As the annealing progresses, the system ideally transitions into the lowest energy state, providing

a solution to the VRP. Recent studies, such as those by Irie et al.[16] and Tambunan et al.[20], demonstrate the viability of quantum annealing in addressing complex VRP instances with dynamic and weighted constraints, highlighting its potential as a tool for optimization in real-world logistics applications.

Given current NISQ hardware limitations, researchers have increasingly focused on combining quantum and classical computing resources to leverage the strengths of both paradigms.

2.7 Hybrid Quantum-Classical Methods and Machine Learning

Beyond pure quantum techniques, researchers increasingly explore hybrid quantum-classical methods that leverage the complementary strengths of both computational paradigms. These approaches aim to circumvent NISQ limitations while still accessing quantum computational advantages.

2.7.1 Variational Quantum Algorithms (VQAs)

Variational Quantum Algorithms, including QAOA and VQE (Variational Quantum Eigensolver), employ a hybrid workflow:

1. Quantum circuit execution prepares parameterized quantum states
2. Measurements extract expectation values
3. Classical optimization adjusts circuit parameters
4. Iteration continues until convergence

Alsaiyari and Felemban (2023)[15] investigated VQAs for VRP, demonstrating that parameterized circuits enable adaptation to problem structure but require careful parameter initialization to avoid poor local minima. Their work emphasized the classical optimization component’s critical role: sophisticated optimizers (COBYLA, Adam, L-BFGS) substantially outperform naive gradient descent.

2.7.2 Quantum Support Vector Machines for VRP

Mohanty et al. (2024)[21] proposed using Quantum Support Vector Machines (QSVMs) for VRP, employing quantum feature maps to embed classical data into high-dimensional Hilbert spaces. The QSVM decision function:

$$f(x) = \text{sgn} \left(\sum_{i=1}^N \alpha_i y_i \langle \phi(x_i) | \phi(x) \rangle + b \right)$$

utilizes quantum kernel evaluation $\langle \phi(x_i) | \phi(x) \rangle$ to compute inner products in feature space, where x is the input data point to classify, x_i are support vectors, $\phi(x) = |\psi(x)\rangle$ represents the quantum feature map that embeds classical data into quantum states, $y_i \in \{-1, +1\}$ are the class labels, α_i are Lagrange multipliers, b is the bias term, N is the number of support vectors, and $\text{sgn}(\cdot)$ is the sign function.

This approach showed promise for route classification problems (identifying high-quality vs. low-quality routes), potentially enabling hybrid methods where quantum classifiers guide classical search heuristics.

2.7.3 Quantum-Inspired Classical Algorithms

Interestingly, quantum algorithm research has inspired improved classical methods. Quantum-inspired algorithms replicate quantum computational principles (superposition-like parallel exploration, interference-like solution amplification) using classical hardware. While not providing true quantum speedups, these algorithms demonstrate that understanding quantum mechanisms yields insights applicable to classical optimization.

A critical challenge across all quantum approaches is the efficient use of limited qubit resources, which has motivated dedicated research on encoding strategies.

2.8 Qubit-Efficient Encoding and Allocation Research

Given NISQ devices' limited qubit counts, recent research emphasizes qubit-efficient problem encodings and optimal qubit allocation strategies.

Balu et al. (2024)[22] proposed a remarkable single-qubit approach for TSP, encoding cost Hamiltonians into amplitude parameters rather than multi-qubit states. While this extreme qubit efficiency limits applicability to very small problems and cannot generalize to full VRP with constraints, it illustrates creative encoding possibilities for resource-constrained quantum devices.

Baek et al. (2023)[23] introduced type-based qubit allocation for first-order quantum programming languages, providing formal methods for analyzing qubit requirements and identifying optimization opportunities. Their work emphasizes that qubit allocation should occur at the algorithmic design stage rather than as an afterthought. Complementing this, Shi *et al.* [24] address distributed quantum computing scenarios where qubits are spread across multiple devices.

Li et al. (2020)[25] developed exact qubit allocation methods for NISQ architectures, accounting for hardware topology constraints. Their algorithms optimally map logical qubits to physical qubits, minimizing SWAP gate overhead—a critical consideration since SWAP gates substantially increase circuit depth and error rates.

Several additional research directions complement the main quantum optimization approaches discussed above.

2.9 Other Important Research on Quantum Optimization

2.9.1 Quantum Algorithms on TSP and VRP

Several works leverage quantum annealing to solve combinatorial problems. Wang *et al.* [26] combine QUBO formulations and Graph Neural Networks to tackle TSP. Kochenberger *et al.* [27] provide a comprehensive Ising formulation of NP problems, useful for annealing frameworks. Earlier work by D-Wave researchers such as Yu *et al.* [28] explore hybrid methods to solve capacitated VRP using annealers.

Several quantum algorithms target TSP directly. The recent work by Surya *et al.* [29] presents an efficient quantum algorithm with promising performance bounds. Another contribution from Younes [22] extends the application to a simplified setting, showing how TSP can be embedded into reduced quantum systems for resource-efficient solving.

2.9.2 Quantum Circuit Optimization

Reducing quantum circuit complexity is crucial for scalable optimization. Liu *et al.* [30] propose an optimization-driven reduction technique to minimize gate depth and number of operations, thereby improving success probability on Noisy Intermediate-Scale Quantum (NISQ) devices.

Having surveyed the breadth of quantum approaches to VRP and related optimization problems, we now synthesize the key findings and identify the research gaps this thesis addresses.

2.10 Summary of Literature Review

The literature reveals several key insights:

1. **Multiple Quantum Approaches Exist:** QAOA, quantum walks, quantum annealing, and hybrid methods each offer distinct advantages and face different challenges for VRP optimization.
2. **Scalability Remains Challenging:** All quantum VRP approaches reported in literature face scalability limitations, typically handling less than 10 nodes maximum on current hardware.
3. **Encoding Strategy Matters:** Theoretical work emphasizes qubit efficiency, but few studies systematically compare encoding strategies experimentally, leaving a gap our research addresses.

4. **Parameter Tuning can be Critical:** QAOA performance depends heavily on circuit depth and parameter optimization quality, yet many studies use arbitrary parameter choices without justification.
5. **Hybrid Methods Show Promise:** Combining quantum and classical components appears more practical near-term than pure quantum approaches, suggesting future research should emphasize hybrid algorithm design.

Our research addresses the identified gaps by providing the first systematic experimental comparison of qubit allocation strategies for quantum VRP solving on real NISQ hardware, with careful parameter optimization and thorough performance characterization.

3 Methodology and Research Design

This section describes the research methodology and implementation details. We begin with our research approach and case study construction using City of Casey waste collection data, then present the VRP model formulation. The core of this section details the QAOA implementation, covering qubit allocation strategies (naive adjacency, logarithmic permutation, and node-visit-time encoding), their respective cost functions, and quantum circuit design. We conclude with computational notes addressing practical implementation considerations.

3.1 Research Approach

Our research employs a mixed-methods approach combining:

- **Theoretical Formulation and Quantum Algorithm Development:** This item deals with the mathematical formulation of qubit allocation strategies, complexity analysis, and theoretical comparison of encoding approaches. This also includes the implementation design of QAOA-based VRP solvers using multiple encoding strategies in quantum programming frameworks (such as Qiskit or PennyLane).
- **Experimental Validation and Comparative Evaluation:** This item involves systematic experiments on a real Quantum Cloud to measure performance across problem instances and parameter settings, followed by a quantitative comparison with classical exact algorithms to establish solution quality baselines.

3.2 Case Study: Waste Collection in the City of Casey

To ground our research in practical applications, we utilize real-world data from the City of Casey waste collection system in Victoria, Australia. This case study provides

realistic problem instances while maintaining manageable problem scales suitable for NISQ devices.

The City of Casey waste collection scenario offers several advantages: **Data Availability:** The City of Casey provides publicly accessible, high-quality datasets through Victoria’s open data portal. This data can represent VRP attributes: depots[3], customers[4, 5], routes[6], and constraints (traffic[7], weather[8], crashes[9]).

Problem Realism: Municipal waste collection represents a genuine VRP application with significant economic and environmental impact. Municipalities worldwide face similar routing challenges, making findings potentially generalizable.

Appropriate Scale: The City of Casey contains 8 waste facilities (depots) and 800+ litter bin clusters. By selecting subsets of bins and facilities, we generate problem instances ranging from 3-6 nodes, which appropriate for current NISQ hardware while retaining real-world problem structure.

Constraint Diversity: Waste collection involves multiple practical constraints (vehicle capacity, time windows, accessibility restrictions) that, while simplified in our initial study, can be incorporated in future work to test encoding robustness.

3.3 VRP Model Construction

To model the VRP, we treat each depot and customer location as a **node** in a graph, with the computed distances representing the **edge weights** between them. We construct the VRP model using three key components: **Depots:** Waste collection facilities serve as depot locations where vehicles begin and end routes. Each depot has:

- Geographic coordinates (latitude, longitude)
- Operating hours (for time window constraints in future work)
- Vehicle capacity (for CVRP extensions)

Customers: Public litter bins and waste collection areas represent customer nodes requiring service. We cluster nearby bins to create aggregate customer locations, reducing problem size while maintaining geographic fidelity. Each customer node has:

- Geographic coordinates
- Service demand (bin capacity/collection volume)
- Accessibility constraints (road access, parking availability)

Routes: The Victoria road network defines feasible vehicle paths between locations. We compute pairwise distances using:

- Euclidean distance (as simplified approximation)
- Road network shortest paths (using Dijkstra’s or A* algorithms)
- Travel time estimates incorporating speed limits and traffic

3.3.1 Problem Instance Generation

From the City of Casey dataset, we generate and choose a number of VRP problem instances with a variety number of nodes (including depot):

Small Instances (3 nodes): Serve as validation cases where exhaustive enumeration provides guaranteed optimal solutions. These instances verify algorithm correctness and establish performance baselines.

Medium Instances (4-5 nodes): Represent the practical operating range for current NISQ devices. These instances allow meaningful quantum-classical comparison while remaining classically tractable for optimal solution computation.

Large Instances (6 nodes): Approach the limits of current quantum hardware, revealing scalability challenges. Classical optimal solutions remain computable via branch-and-bound but require increased computational effort.

Each instance includes:

- Distance matrix: A symmetric $N \times N$ matrix where N is the total number of nodes (customers plus the depot). The element d_{ij} represents the distance/cost between nodes i and j , with the diagonal elements (d_{ii}) being zero.
- Depot designation: Index 0 conventionally represents the depot
- Classical optimal solution: Computed via exact algorithms for benchmark comparison
- Instance metadata: Problem size, geographic region, constraint types

3.3.2 Constraint Handling

While our primary focus addresses the unconstrained VRP formulation (minimize total route distance), the case study infrastructure supports future constraint incorporation:

Traffic Constraints: Traffic volume data can modify edge weights, increasing travel times for congested routes. This reflects real-world conditions where optimal routes avoid high-traffic areas even if geometrically shorter.

Weather Constraints: Rainfall data correlates with reduced travel speeds and increased accident risk. Edge weights can incorporate weather-adjusted travel times.

Road Incidents: Historical crash data identifies high-risk road segments. The model can penalize or prohibit these edges, balancing route efficiency against safety considerations.

Implementation Note: For this thesis, we handle constraints through classical preprocessing, modifying the distance matrix before quantum encoding. Alternative approaches encoding constraints directly into the quantum Hamiltonian remain future work directions.

3.4 QAOA Implementation for VRP

As discussed in Section 2.2, QAOA provides a hybrid quantum-classical framework for combinatorial optimization problems. Here, I describe the specific implementation details for applying QAOA to VRP.

3.4.1 VRP Representation in Quantum Qubits

To apply QAOA to the VRP, we need to represent the problem using quantum qubits. A naive approach is to represent all possible edges between nodes. However, this is not optimal.

- **Graph Representation in Matrix:** As mentioned earlier, we use an $(n+1) \times (n+1)$ matrix to represent the graph.
- **Qubit Allocation:** The key challenge is to efficiently encode the order of customer visits. Using $n + 1$ qubits is insufficient. VRP requires representing not just binary choices, but the order of visits. We found that $2^k > n!$, where k is the number of qubits, is sufficient. This is because there are $n!$ possible permutations of order in n customers. The easiest way to find k is to use the matrix number of element. If $k = (n+1)^2$, 2^k will grow much faster than $n!$ (We can prove this using mathematical ratio with techniques such as stirling's approximation and logarithm)

3.4.2 Qubit Allocation Improvements and Discussion

One of the key challenges in implementing the Quantum Approximate Optimization Algorithm (QAOA) for the Vehicle Routing Problem (VRP) is determining an efficient and scalable strategy for qubit allocation. The choice of encoding directly affects both the computational feasibility and the expressiveness of the quantum circuit. In this study, we evaluate and compare three distinct approaches to qubit allocation:

1. naive adjacency matrix encoding,
2. logarithmic permutation encoding, and
3. node-visit-time encoding.

Each method presents trade-offs in terms of qubit efficiency, implementation complexity, and suitability for enforcing constraints.

Naive Matrix-Based Encoding

The most straightforward approach assigns one qubit per possible directed edge between nodes, resulting in an $N \times N$ grid of qubits for a problem with $N = n + d$ locations (customers and depots). This approach provides a natural representation for the cost function, as distances are encoded directly via edge weights. However, the qubit count grows quadratically with N , which leads to prohibitively large state vectors in simulation and quantum hardware contexts.

Example

Let's assume $N = 6$, which requires 36 qubits and a 2^{36} -dimensional state space. Each qubit q_{ij} represents whether the route includes a directed edge from node i to node j . For example, in a system with four nodes, qubit $q_{1,2}$ encodes whether the vehicle travels from location 1 to location 2. The final route is interpreted as a path composed of edges where the corresponding qubits are in the $|1\rangle$ state.

Logarithmic Permutation Encoding

This approach encodes the entire route as a permutation of customers using the smallest number of qubits possible, i.e., $k = \lceil \log_2(n!) \rceil$. While this method is optimal in terms of qubit usage, it introduces substantial difficulty in mapping quantum bitstrings to meaningful cost evaluations. Constructing a quantum Hamiltonian that accurately reflects VRP constraints under this encoding is highly non-trivial. Furthermore, validity checks and constraint penalties must be implemented as post-processing steps, which diminishes the effectiveness of in-circuit cost optimization.

Example

With $n = 3$ customers, there are $3! = 6$ possible routes, and $k = \lceil \log_2(6) \rceil = 3$ qubits are sufficient. Each measured bitstring from the quantum circuit (e.g., 010 or 101) is interpreted as an integer index into a precomputed list of valid permutations. A bitstring like 011 might correspond to the customer sequence $[1, 3, 2]$, yielding the route $[0, 1, 3, 2, 0]$ with the depot appended at start and end.

Node-Visit-Time Encoding (Proposed Hybrid Method)

To balance qubit efficiency and route validity, we introduce a hybrid approach wherein each customer is represented by a fixed-size binary register that denotes their position in the route (i.e., visit time). This method requires $n \cdot \lceil \log_2(N) \rceil$ qubits and allows for structured constraint enforcement. Each route is reconstructed by sorting customers based on their encoded visit times. Invalid configurations, such as duplicate positions or unassigned slots, are penalized through additional constraint terms in the Hamiltonian.

Example

Using the same example, with $n = 3$ customers and $N = 4$ total nodes, each customer is assigned a 2-qubit register to represent their visit time. If customer 1 is encoded by qubits q_0, q_1 , and their values are 01, this indicates that customer 1 is visited second in the route. A full bitstring such as 01101011 might be interpreted as: customer 1 at position 1, customer 2 at position 2, customer 3 at position 3, leading to a route of $[0, 1, 2, 3, 0]$. Duplicate visit times can be easily detected and penalized by counting repeated integers in the decoded visit-time list.

Comparison

The following table summarizes the characteristics of the three qubit allocation strategies:

Encoding Method	Qubit Count	Num of Qubit	Cost Function	Simplicity
Naive Matrix (NxN)	N^2	Large		Simple
Logarithmic Permutation	$\lceil \log_2(n!) \rceil$	Small	Complex (impractical)	
Node-Visit-Time	$n \cdot \lceil \log_2(N) \rceil$	Moderate		Fair

In conclusion, while the naive approach is conceptually straightforward and suitable for initial experimentation, it becomes infeasible for larger instances. The logarithmic method offers minimal qubit usage but at the cost of interpretability and constraint encoding. The node-visit-time encoding offers a middle ground, enabling manageable circuit sizes while supporting effective constraint modeling within the QAOA framework.

3.4.3 Cost Function for Naive Matrix-Based Encoding

The cost function in our QAOA implementation of the VRP is designed to minimize the total travel distance while ensuring the solution represents a valid route. This involves two main components:

1. **Distance Cost::** This term represents the total travel distance, which we aim to minimize.
2. **Constraint Penalties:** These terms penalize invalid routes that violate the VRP's constraints, such as visiting a customer more than once or not visiting a customer at all.

Let's consider representing the connections between locations using binary variables x_{ij} , where $x_{ij} = 1$ if we travel from location i to location j , and $x_{ij} = 0$ otherwise. Let d_{ij} be the distance between location i and location j . We can formulate the cost function as:

$$C(x) = Z_1 \sum (d_{ij} \cdot x_{ij}) + Z_2 \sum \left(\sum x_{ij} - 1 \right)^2 + Z_3 \sum \left(\sum x_{ji} - 1 \right)^2$$

where Z_1, Z_2, Z_3 are penalty parameters. In most cases, it makes sense to set $Z_2 = Z_3$.

3.4.4 Cost Function for Logarithmic Permutation Encoding

In the logarithmic permutation encoding approach, the solution space is limited to the factorial number of valid customer permutations. Each quantum state corresponds to an index in the set of valid permutations of customers. Therefore, cost evaluation must operate over the decoded bitstring, treating it as a lookup into a predefined set of valid routes.

Formally, let $|b\rangle$ be a basis state in the computational basis of the k -qubit system. This bitstring is interpreted as an integer index. Let $P = \{r_1, r_2, \dots, r_{n!}\}$ be the ordered list of all valid customer visit sequences (without the depot). Each state $|b\rangle$ corresponds to a specific route r_i , where $i = \text{int}(b)$.

To compute the travel cost, the full route is formed by adding the depot at the beginning and end of the customer sequence: $[0, r_i[1], r_i[2], \dots, r_i[n], 0]$, and the total distance is calculated based on this path.

The cost function is defined as:

$$C(|b\rangle) = \begin{cases} \text{TotalDistance}(r_i), & \text{if } i < n! \\ Z_4, & \text{otherwise (invalid index)} \end{cases}$$

Here, $\text{TotalDistance}(r_i)$ is the sum of distances along the route that starts and ends at the depot, and Z_4 is a large penalty applied to any bitstring that refers to an invalid index (i.e., $i \geq n!$).

This cost function is difficult to implement directly as a Hamiltonian because it requires knowing the total travel cost for each possible bitstring in advance. As a result, it typically requires classical pre-processing and post-sampling evaluation. While this encoding uses very few qubits, it trades off ease of modeling and scalability. In the case of VRP, generating the full list of valid routes and their distances is already computationally expensive and similar to classical method problem-solving.

3.4.5 Cost Function for Node-Visit-Time Encoding

In the node-visit-time encoding, each customer is assigned a binary register that represents their position in the route, also called their “visit time.” The complete route is reconstructed by sorting all customers based on their assigned visit times. The vehicle is assumed to start and end at the depot (location 0), so the final route becomes: depot $\rightarrow$ customer₁ $\rightarrow$ customer₂ $\rightarrow \dots \rightarrow$ customer _{n} $\rightarrow$ depot.

Let each customer i be assigned a visit time value v_i decoded from its corresponding group of qubits. Once all v_i values are obtained, we sort the customers by these values

to determine the order in which they are visited.

The cost function consists of three main components:

1. **Travel Distance (Objective):** This is the total distance traveled through the full route. If the sorted customer order is $[c_1, c_2, \dots, c_n]$, the route is $[0, c_1, c_2, \dots, c_n, 0]$. The total travel cost is:

$$C_{\text{dist}} = d_{0,c_1} + \sum_{i=1}^{n-1} d_{c_i,c_{i+1}} + d_{c_n,0}$$

where $d_{i,j}$ is the distance between location i and location j .

2. **Uniqueness Constraint (No Duplicate Positions):** We must ensure that each customer is assigned a unique visit time. If two customers are assigned the same position (e.g., both want to be second in the route), the solution is invalid. We penalize such cases by:

$$C_{\text{dup}} = Z_5 \cdot \sum_{\text{time}=1}^n (\text{count}(\text{time}) - 1)^2$$

where $\text{count}(\text{time})$ is the number of customers assigned to that visit time. If all visit times are unique, this penalty is zero.

3. **Out-of-Range Constraint (Invalid Bitstrings):** Since each visit time is represented by a binary number, some combinations may produce values outside the valid range $[1, n]$. For example, if 2 bits are used per customer (allowing values 0 to 3), then with 3 customers, a value of 0 or 4 is invalid. We penalize any customer with an out-of-range visit time:

$$C_{\text{out}} = Z_6 \cdot \sum_{i=1}^n \text{Invalid}(v_i)$$

where $\text{Invalid}(v_i)$ is 1 if v_i is less than 1 or greater than n , and 0 otherwise.

The final cost function is the combination of the three components:

$$C = C_{\text{dist}} + C_{\text{dup}} + C_{\text{out}}$$

This formulation makes it possible to balance between minimizing the route length and enforcing valid and unique visit assignments. Some parts of the cost function can be encoded directly into the quantum Hamiltonian, while others (such as the duplicate and out-of-range penalties) may be more efficiently applied in a classical post-processing step.

3.4.6 Qubit Allocation for Depots and Vehicles

The choice of encoding depots and vehicles to qubits directly affects both the computational feasibility and the performance of the quantum circuit. In this study, we evaluate and compare three distinct approaches to qubit allocation:

1. One-Hot Encoding,
2. Separator Combinatorial Encoding, and
3. Hybrid/Optimized Encoding.

Each method presents trade-offs in terms of qubit efficiency, implementation complexity, and suitability for enforcing constraints.

One-Hot Encoding

One-hot encoding is a widely used approach in classical optimization problems and involves assigning a separate qubit for each possible state or assignment. In the context of VRP, each node (depot, vehicle, and customer) is represented using a binary qubit string where only one of the bits is set to 1, corresponding to the active assignment, while all others are set to 0. This approach guarantees a straightforward representation of the problem but requires a large number of qubits as the number of locations (depots, vehicles, and customers) grows.

Qubit Calculation

Consider a scenario with d depots, v vehicles, and n customers. - The qubits for depot encoding would require:

$$Q_{\text{depot}} = d$$

- The qubits for vehicle encoding would require:

$$Q_{\text{vehicle}} = v$$

- The qubits for depot-to-vehicle assignment would require:

$$Q_{\text{depot-vehicle}} = v \cdot d$$

- The qubits for vehicle-to-customer assignment would require:

$$Q_{\text{vehicle-customer}} = v \cdot n$$

Thus, the total qubit count for One-Hot Encoding is:

$$Q_{\text{total}} = Q_{\text{depot}} + Q_{\text{vehicle}} + Q_{\text{depot-vehicle}} + Q_{\text{vehicle-customer}}$$

$$Q_{\text{total}} = d + v + v \cdot d + v \cdot n$$

Separator Combinatorial Encoding

The separator combinatorial method is an alternative encoding strategy where separators (such as ‘-1’) are used to demarcate distinct groups or transitions in the routing process. This approach reduces the number of qubits by leveraging combinatorial structures rather than representing each possible state directly. It is particularly useful when working with problems that have natural divisions, such as a depot and its associated vehicles or customers.

Example

For the same 3 depots and 5 vehicles setup, a separator combinatorial approach could encode the depot-to-vehicle assignment as a binary index mask, where each bit represents whether a vehicle is assigned to a particular depot. For example, the depot-to-vehicle encoding could use ‘00011’ to indicate that Vehicles 1 and 2 are assigned to Depot 1, ‘00110’ for Depot 2, and ‘01000’ for Depot 3. This reduces the total number of qubits needed by representing groupings and transitions more compactly.

Qubit Calculation

For Separator Combinatorial Encoding, the qubit count is as follows: - Depot-to-vehicle encoding requires:

$$Q_{\text{depot-vehicle}} = \lceil \log_2 [(v + (d - 1))!] \rceil$$

- Vehicle-to-customer encoding requires:

$$Q_{\text{vehicle-customer}} = \lceil \log_2 [(n + (v - 1))!] \rceil$$

Thus, the total qubit count for Separator Combinatorial Encoding is:

$$Q_{\text{total}} = Q_{\text{depot-vehicle}} + Q_{\text{vehicle-customer}}$$

$$Q_{\text{total}} = \lceil \log_2 [(v + (d - 1))!] \rceil + \lceil \log_2 [(n + (v - 1))!] \rceil$$

Hybrid/Optimized Encoding

The hybrid/optimized encoding method aims to combine the benefits of both one-hot encoding and combinatorial encoding. Instead of directly encoding all possible states for each component (depot, vehicle, customer), this method uses a more efficient representation by assigning a binary register to each customer or vehicle that reflects their position or assignment in the route. This allows for a more compact encoding of the routing problem, reducing the qubit count while still enabling effective constraint enforcement.

Example

Consider a scenario with 3 depots, 5 vehicles, and 4 customers. The hybrid encoding would allocate:

- 2 qubits for the depot encoding (for 3 depots),
- 3 qubits for the vehicle encoding (for 5 vehicles),
- 4 qubits for the depot-to-vehicle assignment (15 possible arrangements for 3 depots and 5 vehicles),
- 5 qubits for each vehicle-to-customer assignment (20 possible arrangements for 5 vehicles and 4 customers).

Qubit Calculation

Thus, the total qubit count for Hybrid/Optimized Encoding is:

$$Q_{\text{total}} = \lceil \log_2(d) \rceil + \lceil \log_2(v) \rceil + \lceil \log_2(d \cdot v) \rceil + \lceil \log_2(v \cdot n) \rceil$$

Comparison

The following table summarizes the characteristics of the three qubit allocation strategies:

Encoding Method	Num of Qubits	Cost Function Simplicity
One-hot encoding	Large	Simplest
Separator Combinatorial Encoding	Small	Complex
Hybrid/Optimized Encoding	Moderate	Fair

In conclusion, while one-hot encoding is conceptually simple and easy to implement, it becomes inefficient as the problem size increases, leading to high qubit counts. The separator combinatorial method reduces the qubit count by encoding the problem in a more compact manner but may introduce complexity in handling constraints. The hybrid/optimized encoding strikes a balance between qubit efficiency and constraint handling, making it a promising candidate for solving larger VRP instances in a quantum setting.

3.4.7 Quantum Circuit and Hamiltonian

The cost function $C(x)$ is translated into a quantum Hamiltonian H_C by replacing the binary variables x_{ij} with Pauli-Z operators. The quantum circuit alternates between the cost Hamiltonian and a mixing Hamiltonian, usually using a rotation gate. The depth p determines the number of layers. The unitaries used are:

$$U_C(\gamma) = e^{-i\gamma H_C}, \quad U_M(\beta) = e^{-i\beta H_M}$$

The γ (gamma) parameter as well as β (beta) in the QAOA circuit control the evolution under the cost and mixing Hamiltonians, respectively. We can start with random initial values for these parameters and then optimize them using a classical optimizer.

In an n -depth Hamiltonian, the circuit structure is illustrated in Figure 2:

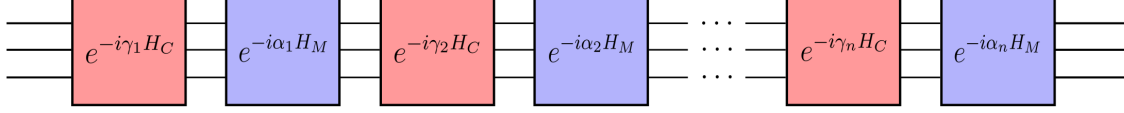


Figure 2: QAOA circuit structure with more than one depth ($p > 1$): Hadamard initialization, alternating cost and mixer layers.

3.4.8 Addressing Constraints

This study addresses real-world constraints in the Vehicle Routing Problem (VRP), such as traffic congestion, weather conditions, and road accidents, through classical preprocessing techniques. These constraints are not directly encoded into the quantum model. Instead, they are used to modify the graph structure that is later passed to the quantum optimization algorithm. For example, areas with heavy traffic or frequent accidents are assigned higher edge weights to reflect longer travel times or added risks.

This hybrid approach reduces the complexity of the quantum circuit while still incorporating practical considerations. Given current limitations in quantum hardware, such as limited qubit count and short coherence time, it is more practical to handle dynamic, real-world factors using classical methods before quantum encoding. Once the graph has been adjusted to include these constraint-aware edge costs, it is converted into a QUBO or Hamiltonian format suitable for quantum algorithms like QAOA.

Although the primary focus is on quantum optimization, several stages (including data preparation, constraint integration, and solution verification) remain within the classical domain. This results in a hybrid quantum-classical workflow, where the classical layer prepares a realistic problem representation, and the quantum layer concentrates on exploring optimal or near-optimal solutions.

3.5 Other Notes

3.5.1 Memory Problem

We encountered a significant memory problem when attempting to simulate the QAOA circuit for even a small VRP instance (5 customers, 1 depot). The PennyLane simulator attempts to store the entire quantum state vector, requiring 2^k complex amplitudes. For $n = 5$, $k = 36$, which leads to:

$$2^{36} \cdot 16 \text{ bytes} = 2^{40} \text{ bytes} = 1 \text{ Terabyte}$$

Therefore, using $(n + 1)^2$ qubits would have resulted in a memory requirement of exactly 1 Terabyte. This is of course impractical. So we need to find another k where $2k > n!$. Which is why we can use another way of encoding such as logarithmic permutation on separator combinatorial encoding.

3.5.2 Comparison of Operation Number (Classical vs. Quantum)

- **Classical:** Classical algorithms for the VRP, such as exhaustive search, require exploring all possible permutations, resulting in approximately $(n + 1)!$ operations for n customers.
- **Quantum:** QAOA aims to find approximate solutions with potentially fewer operations, but the exact reduction depends on the problem instance and QAOA parameters. The higher the depth p , the higher the complexity. The circuit qubit requirements and circuit complexity will also increase. The total quantum gate operation is estimated as $s \cdot p \cdot (n + 1)^2$ operations (which is less than $(n + 1)!$), with controlled iterations s and circuit depth p .

3.5.3 Using v Vehicles and d Depots

Extending to multiple depots and vehicles, we get:

$$T(n, d, v)_{\text{Classical}} \approx (n + d)! \cdot v^n$$

$$T(n, d, v, p, s)_{\text{Quantum}} \approx s \cdot p \cdot v \cdot (n + d)^2$$

- **Classical:** With v vehicles and d depots, the classical problem's complexity increases significantly, as we need to consider all possible assignments of customers to vehicles and depots.
- **QAOA:** QAOA can potentially handle this increased complexity, but the qubit requirements and circuit complexity will also increase. We would need to encode

the vehicle assignments and depot selections into the quantum state.

4 Experimental Directions and Setup

This section presents the experimental framework and setup for evaluating quantum VRP optimization. We first discuss the experimental directions and justifications, including comparisons between VRP solvers and research questions guiding our experiments. We then detail the experimental setup covering problem instances, quantum hardware configuration, and performance metrics. The experiments systematically compare naive adjacency and node-visit-time encodings across multiple parameters and instances.

All experimental code and data are documented in the GitHub repository for traceability and reproducibility.¹

4.1 Experimental Directions and Some Justification

4.1.1 Comparison of VRP Solvers: Naive vs. Google OR-Tools

This analysis compares a naive brute-force approach to solving the Vehicle Routing Problem (VRP) against Google’s OR-Tools. The goal is to highlight the trade-offs between finding a provably exact solution and achieving a near-optimal solution efficiently.

The fundamental difference between the two solvers lies in their approach to optimality.

- **Naive Solver (Brute-Force):** This method explores every possible permutation of the path, guaranteeing that it will find the absolute shortest route. In a **theoretical sense**, this solution is 100% optimal because it has exhaustively proven that no shorter path exists. However, this approach is computationally expensive and becomes infeasible as the number of locations increases due to the factorial growth of possible paths.
- **Google OR-Tools:** OR-Tools uses a combination of heuristics and metaheuristics, such as **Guided Local Search**, to find a high-quality solution within a reasonable time. It starts with an initial feasible route and iteratively improves it. In a **practical sense**, the solution provided by OR-Tools is often considered optimal because the difference from the true optimum is negligible (e.g., 10^{-4} km), and the computational speed-up is significant. The slight discrepancy arises from its reliance on approximation algorithms rather than an exhaustive search.

Based on some of our data, we can directly compare the performance of both solvers for the same VRP instance with one sample of waste facility at a specific location.

¹<https://github.com/drestanto/quantumvrp>

Naive Solver

The naive solver found the following distance (the numbering is just to show that there are discrepancy between the two solvers):

- **Optimal Path:** $5 \rightarrow 22 \rightarrow 57 \rightarrow 87 \rightarrow 70 \rightarrow 48 \rightarrow 5$
- **Minimum Distance:** 26.039 km
- **Processing Time:** 0.0006 seconds

This result represents the true, mathematically proven shortest path.

OR-Tools Solver

OR-Tools, given a 10-second time limit, found a different path with a very similar distance:

- **Optimal Path:** $5 \rightarrow 48 \rightarrow 70 \rightarrow 87 \rightarrow 22 \rightarrow 57 \rightarrow 5$
- **Minimum Distance:** 26.035 km
- **Processing Time:** 10.0193 seconds

Although the paths are different, the distance values are practically identical, with a minute difference of 0.004 km (4 meters).

Google's OR-Tools is designed for scalability and speed. It finds a good solution quickly, especially for large, complex problems where a brute-force approach would be impossible. The solver might not find the exact optimum for several reasons:

1. **Limited Search Time:** A constrained time limit prevents the solver from exploring the entire solution space. The 10-second limit in this example is sufficient for a good solution but not necessarily for a provably optimal one.
2. **Heuristic-Based Search:** The use of metaheuristics like **Guided Local Search** means the algorithm might get stuck in a local optimum: a solution that is better than its immediate neighbors but not the best overall solution.
3. **Initial Solution:** The quality of the final solution can be influenced by the initial random route, which might guide the search away from the absolute best path.

Conclusion

The comparison demonstrates the trade-off between **precision** and **performance**. The naive solver is an effective approach for its ability to find the absolute best solution, but its practical application is limited to very small problems. Google's OR-Tools offers a scalable and efficient way to find a high-quality, near-optimal solution for real-world VRP challenges, where a tiny difference in distance is negligible compared to the time saved.

4.1.2 Research Directions on These Experiments

Several research directions can be derived from this experimental implementation in Quantum Optimization for VRP.

1st Option: Qubit Efficient VRP Solver

Traditional quantum VRP solvers often use a straightforward but inefficient qubit allocation method. The common approach allocates N^2 qubits, where N is the number of customers. This is done to encode both the nodes and the sequence of their visits.

Let's assume $N = 6$, which requires 36 qubits and a 236-dimensional state space. Each qubit q_{ij} represents whether the route includes a directed edge from node i to node j . For example, in a system with four nodes, qubit $q_{1,2}$ encodes whether the vehicle travels from location 1 to location 2. The final route is interpreted as a path composed of edges where the corresponding qubits are in the $|1\rangle$ state.

Research Direction

A promising research direction is to adopt a **node-visit-time encoding**, a more qubit-efficient method inspired by research like the work from [31]. This approach aims to significantly reduce the number of qubits required, making it feasible to solve larger VRP instances on current and near-future quantum computers.

2nd Option: VRP Preprocessing Challenge

VRPs typically involve spatial points (depots, customers) and road networks. A crucial preprocessing step is to construct a complete graph from these points by finding the shortest path between all pairs of locations on the road network. There are two primary classical approaches for this:

1. **All-Pairs Shortest Paths (APSP):** Calculate the shortest paths for all pairs of nodes in the road network and then extract the subset relevant to the depots and customers. Best classical algorithms for this include Floyd-Warshall or min-plus matrix multiplication.
2. **Single-Source Shortest Path (SSSP):** Calculate the shortest paths from each depot and customer to all other depots and customers. A* is a common algorithm for this.

Research Direction

A research direction here is to explore the use of **quantum distributed algorithms** [32] to compute APSP. The findings can be compared with the SSSP Bellman-Ford algorithm, which could potentially be accelerated using **quantum dynamic programming** [33].

3rd Option: Managing Constraints in VRP

Constraints (e.g., traffic, time windows, vehicle capacity) are central to VRP. A naive approach is to handle these during preprocessing by modifying graph weights. For example, a longer but less congested road might be given a lower weight than a shorter but traffic-heavy one. This results in a modified graph that can be solved with standard VRP methods.

Research Direction

An alternative, inspired by guided local search methods, is to manage constraints iteratively during the optimization process. In the context of quantum computing, a research direction is to explore how to incorporate constraints into **Quantum Approximate Optimization Algorithm (QAOA)**.

- **Penalty in Cost Hamiltonian:** The conventional approach is to add a penalty term to the cost Hamiltonian for states that violate constraints [34].
- **Constraint in Mixer Hamiltonian:** A more advanced approach is to design a mixer Hamiltonian that only allows transitions between states that satisfy the constraints.
- **Adaptive QAOA Steps:** Another method could involve adding more QAOA steps to an iteration, allowing the algorithm to satisfy constraints dynamically as it converges to an optimal solution [35].

4th Option: Swap Test for Clustering

VRPs can involve a vast number of data points, making clustering a necessary preprocessing step to reduce the problem size. Classical clustering algorithms like k-means can be computationally intensive, with a complexity of $O(N \cdot k)$ and a faster version using k-d trees at $O(N \log N)$. HPC research has further parallelized k-d trees to achieve $O(N \log N/p)$ speedup.

Research Direction

A quantum research direction is to use **angle encoding** to represent 2D spatial points as a 2-qubit quantum state. Once the data is encoded in this manner, a **quantum swap test** can be used to calculate the inner product between different data points. This allows for a form of **quantum parallelism** [36] to cluster the data more efficiently than classical methods. This could be a powerful tool for reducing the complexity of large-scale VRPs.

Conclusion on Research Directions

Two options, 3rd and 4th, were eliminated initially because their practicality would require an additional, separate research literature review and methodology, making them impractical given the time limit of this study.

For the 2nd research direction option, focusing on quantum-accelerated VRP preprocessing. This address All-Pairs Shortest Path (APSP) and Single-Source Shortest Path

(SSSP) problems but is deemed impractical. The following points outline the key issues:

- **Theoretical Quantum Algorithms:** The proposed quantum distributed algorithms for APSP, based on research by Izumi and Le Gall [37], are considered purely theoretical and lack a practical implementation in current quantum settings [37]. Similarly, the idea of using quantum dynamic programming for the Bellman-Ford algorithm is based on a paper by Caroppo et al. [33], which frames the approach as an open problem rather than a practical solution framework [33]. This makes these methods irrelevant for a research project that requires a tangible, implementable solution.
- **Algorithmic Obsolescence:** Current classical algorithms are advancing rapidly. The idea of using Grover’s algorithm within Dijkstra’s to find minimum values is based on a greedy, probabilistic approach. This method is already at risk of becoming outdated. A new deterministic SSSP algorithm [38], which is faster than Dijkstra and uses a priority queue instead of a matrix data structure, has recently been published. This highlights the need to constantly keep up with fast-paced classical research, making a quantum approach potentially obsolete before it’s developed.
- **Effective Classical Alternatives:** Highly effective classical solutions already exist. For example, Google’s OR-Tools provides a fast VRP solution using heuristic methods, which are already considered excellent in this domain. The availability of such robust classical tools reduces the need for a novel, and currently impractical, quantum approach.

4.1.3 Optimizers Justification in QAOA for the VRP

The Quantum Approximate Optimization Algorithm (QAOA) is a hybrid quantum-classical algorithm that finds approximate solutions to complex combinatorial optimization problems. A major application for QAOA is the **Vehicle Routing Problem (VRP)**, a logistics challenge that seeks the most efficient routes for a fleet of vehicles. The VRP is computationally difficult, with complexity that grows exponentially with the problem size.

QAOA addresses this by translating the VRP into a quantum operator called a **cost Hamiltonian**. The lowest energy state of this Hamiltonian corresponds to the optimal VRP solution. The process is executed in a feedback loop:

- A quantum computer prepares and measures a special quantum state.
- A classical computer uses these measurement results to adjust the quantum circuit’s parameters to improve the solution.

This write-up focuses on the crucial role of the **optimizer** in the classical part of this loop. The optimizer’s purpose is to find the optimal values for the QAOA’s two sets of parameters: β and γ . These angles, which control the application of the mixer and cost Hamiltonians, are essential for the quality of the final solution.

Optimizer Explanation

Optimizers are broadly categorized into two types: classical and quantum.

- **Classical Optimizers:** These are algorithms that run on a classical computer to control the quantum part of the QAOA.
- **Quantum Optimizers:** A newer class of optimizers designed specifically to leverage quantum information to better navigate the unique landscape of quantum state spaces. An example is the **Quantum Natural Gradient (QNG)**.

Classical optimizers can be further divided into two groups: **gradient-based** and **gradient-free** methods.

- **Gradient-Based Optimizers:** These methods use the gradient (the direction of the steepest descent) to guide their search for the minimum. Think of it as a hiker always taking a step in the steepest downward direction to get to the bottom of a valley.
- **Gradient-Free Optimizers:** These methods do not use gradient information. Instead, they probe the surrounding landscape by making small changes to parameters and checking if the solution improves. This is like a person in a foggy valley, trying a few steps in different directions and choosing the one that leads down.

Examples of well-known optimizers include:

- **Gradient-Based:** SPSA (Simultaneous Perturbation Stochastic Approximation), Adam, and BFGS (Broyden–Fletcher–Goldfarb–Shanno algorithm).
- **Gradient-Free:** COBYLA (Constrained Optimization by Linear Approximation) and Nelder-Mead.

Optimizer Choosing

Choosing an optimizer for QAOA on VRP is a practical decision that depends on the problem size and available hardware. A classical optimizer is typically used today because quantum optimizers are still an area of active research.

For QAOA applied to VRP, a **gradient-free** method is often a more useful choice.

- **Why a Gradient-Free Optimizer?**

The QAOA parameter landscape is highly non-convex and noisy with many local minima. Attempting to compute a reliable gradient on a Noisy Intermediate-Scale

Quantum (NISQ) device is challenging. Gradient-free methods are more robust to this noise and are less likely to get stuck in barren plateaus, where gradients vanish.

- **Why COBYLA?**

COBYLA is a good choice for smaller VRP instances because it is designed to handle non-linear optimization with constraints, which is how the VRP is encoded in the QAOA circuit. It is a derivative-free method, making it well-suited for noisy environments.

- **Drawbacks of COBYLA:**

The main drawback of COBYLA is that it can be **slow to converge**. As the number of parameters (determined by the QAOA circuit depth) increases, the search space becomes much larger. Without the guidance of a gradient, finding the optimal solution can become very time-consuming.

4.2 Experimental Setup

The success of quantum optimization for VRP will be based on effectively translating the classical problem into a quantum language that can be processed by a quantum computer. This involves a clear encoding scheme and a well-defined cost function. This part will detail the comprehensive data processing pipeline established for a Vehicle Routing Problem (VRP) project, leading to the preparation of input data for both classical and quantum optimization algorithms.

4.2.1 Data Preprocessing)

Data processing is organized into a hierarchical folder structure, reflecting a clear progression from raw inputs to highly structured graph data suitable for VRP solvers. The primary goal is to derive distance matrices for clusters of locations (waste facilities, litter bins, collection areas).

Stage 1: 0_raw data: This initial stage contains the raw, unprocessed CSV datasets. These files serve as the foundational input for the entire pipeline.

- `clean.py`: Script likely used for initial data cleaning operations.
- `public-litter-bins.csv`: Raw data pertaining to public litter bin locations.
- `road-responsibility.csv`: Raw data related to road responsibility, potentially for routing constraints.
- `waste-collection-area.csv`: Raw data for waste collection areas.

- `waste-facility-locations.csv`: Raw data detailing waste facility locations.

Stage 2: 1_cleaned data: Following the initial cleaning, this stage stores the refined datasets. The cleaning process typically involves handling missing values, correcting inconsistencies, and standardizing formats.

- `isolate.py`: Script likely used for further data refinement, possibly isolating specific regions or types of data.
- Cleaned versions of the raw CSVs (e.g., `public-litter-bins-clean.csv`).

Stage 3: 2_isolated data: This stage contains data that has undergone isolation or spatial filtering, focusing on relevant geographical areas or subsets of data.

- `cluster.py`: Script responsible for grouping or clustering the isolated data points.
- Spatially processed versions of the cleaned CSVs (e.g., `public-litter-bins-clean-spatial.csv`).

Stage 4: 3_clustered data: This stage holds the results of the clustering process, typically in JSON format, indicating how various locations are grouped.

- `complete_grapher.py`, `complete_grapher_legacy.py`: Scripts used to generate complete graphs (where every node is connected to every other node) and calculate distances within these clusters.
- Clustering results in JSON format (e.g., `public_litter_bins_to_waste_collection_areas_clustering.json`), likely containing node IDs and their cluster assignments.

Stage 5: 4_completed graph data: This final data processing stage contains the core input for VRP solvers: complete graphs with pre-calculated distance matrices for each cluster.

- `cluster_complete_graphs.json`, `cluster_complete_graphs_legacy.json`: These JSON files contain the distance matrices for various clusters, ready for VRP optimization.
- `naive_vrp.py`: A classical brute-force VRP solver that utilizes these `cluster_complete_graphs.json` files as its primary input to find optimal routes.
- `lookuper.py`, `sample_digestor.py`: Auxiliary scripts for data lookup or digestion, supporting the overall data flow.

4.2.2 VRP Instance Details

The implementation utilizes VRP data from a JSON file (`cluster_complete_graphs.json`), focusing on a `waste_facility` with ID 5. The dataset provides a comprehensive distance matrix between various geographical nodes. Key parameters extracted for the current problem instance include:

- **Nodes:** [5, 22, 48, 57, 70, 87] (the numbering shows the location IDs)
- **Depot:** 5 (This is the central waste facility)
- **Customers:** [22, 48, 57, 70, 87] (These are the waste collection areas to be visited)
- **Total Nodes (N):** 6 (Depot + Customers)
- **Number of Customers (n):** 5

4.2.3 Node-Visit-Time Encoding

The chosen encoding strategy is the "Node-Visit-Time Encoding", which represents the sequence of customer visits. Instead of directly encoding paths, this method assigns a discrete "time slot" or "order" to each customer node.

To represent these time slots using qubits, we determine the number of bits required for each node. If there are N total nodes (depot + customers), then each customer's visit time can range from 0 to $N - 1$. The number of bits needed to uniquely identify these N possible time slots is $\lceil \log_2(N) \rceil$.

For our instance:

- **Bits per node ($\lceil \log_2(6) \rceil$):** 3 bits are required to represent time slots from 0 to 5. For example, $000_2 = 0$, $001_2 = 1$, ..., $101_2 = 5$.
- **Total Qubits required:** Since there are 'num_customers' customers and each needs 'bits_per_node' bits to encode its visit time, the total number of qubits is $\text{Number of Customers} \times \text{Bits per node} = 5 \times 3 = 15$ qubits.

A quantum state (bitstring) of these 15 qubits is interpreted by grouping the bits for each customer. For example, if we have customers A, B, C, D, E and each requires 3 bits, a 15-bit string 'b14 b13 ... b0' would be parsed as:

- Bits [2:0] for Customer A's visit time.
- Bits [5:3] for Customer B's visit time.
- ...and so on.

These decoded integer visit times are then used to sort the customers, forming the sequence of visits. The full route starts at the depot, visits customers in the determined order, and returns to the depot.

4.2.4 Cost Function Implementation

The objective of QAOA is to minimize a classical cost function that measures the "goodness" of a solution. For this VRP, the cost function for a decoded route encompasses two main parts:

1. **Total Distance Cost:** This is the sum of Euclidean (or road network) distances between consecutive nodes in the generated route, starting from the depot, visiting all customers, and returning to the depot. The distances are retrieved from the pre-calculated 'distance_matrix.km'.
2. **Penalty Costs:** These are added to penalize routes that violate VRP constraints, making them less favorable for the optimizer.
 - **Duplicate Visit Penalty** (PENALTY_DUPLICATE_VISIT = 1000.0): This large penalty is applied if two or more customers are assigned the exact same visit time. In a valid VRP solution, each customer must be visited exactly once, implying unique visit times for distinct customers.
 - **Out-of-Range Penalty** (PENALTY_OUT_OF_RANGE = 1000.0): This penalty applies if a decoded visit time for any customer falls outside the valid range of 0 to $N - 1$. This ensures that the quantum measurements yield meaningful time slot assignments.

The optimizer seeks to find quantum parameters that minimize this total cost, thereby favoring routes with minimal distance and no constraint violations.

4.2.5 Conceptual Link to VRP Cost Function (Implicit Hamiltonian)

QAOA is a hybrid quantum-classical algorithm. It uses a quantum circuit to prepare a state that encodes the solution to an optimization problem, and a classical optimizer to iteratively refine the parameters of this quantum circuit. In a fully formalized QAOA, the VRP is typically first converted into a Quadratic Unconstrained Binary Optimization (QUBO) problem or an Ising Hamiltonian. This Hamiltonian, H_C , represents the problem's objective and constraints. For example:

$$H_C = \sum_{i < j} J_{ij} Z_i Z_j + \sum_i h_i Z_i$$

where Z_i is the Pauli Z operator on qubit i , and J_{ij} and h_i are coefficients derived from the VRP distances and penalties.

The QAOA then applies a unitary operator $U_C(\gamma) = e^{-i\gamma H_C}$, which is approximated using a sequence of quantum gates corresponding to the terms in H_C . Similarly, a mixer Hamiltonian, often $H_M = \sum_i X_i$, is used to generate $U_M(\beta) = e^{-i\beta H_M}$, which encourages exploration of the solution space.

4.2.6 QAOA Circuit Construction (`create_qaoa_circuit`)

The current implementation uses a general QAOA circuit structure for ‘p’ layers (depth ‘p=2’ in this instance). The parameters γ and β are the angles for quantum gates.

1. **Initial Layer:** A Hadamard gate (H) is applied to each of the ‘num_qubits’. This creates an initial uniform superposition state over all possible bitstrings, ensuring that all potential solutions are explored.

$$|\psi_0\rangle = \bigotimes_{i=0}^{N_q-1} H |0\rangle = \frac{1}{\sqrt{2^{N_q}}} \sum_{x \in \{0,1\}^{N_q}} |x\rangle$$

2. **Cost Layer ($U_C(\gamma)$):** This layer implements the phase rotations corresponding to the problem’s cost Hamiltonian. In this simplified implementation, we use:

- **Single-qubit R_Z gates:** $R_Z(2\gamma_k)$ applied to each qubit i . The $R_Z(\theta)$ gate implements a rotation around the Z-axis of the Bloch sphere by an angle θ :

$$R_Z(\theta) = \exp\left(-i\frac{\theta}{2}Z\right) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}$$

These gates are typically used to encode single-qubit terms (Z_i) of the cost Hamiltonian.

- **Two-qubit R_{ZZ} gates:** $R_{ZZ}(2\gamma_k)$ applied to all pairs of qubits (i, j) . The $R_{ZZ}(\theta)$ gate implements a rotation around the ZZ-axis for two qubits:

$$R_{ZZ}(\theta) = \exp\left(-i\frac{\theta}{2}Z \otimes Z\right)$$

These are used to encode two-qubit interaction terms ($Z_i Z_j$) of the cost Hamiltonian.

The ‘gamma.k’ parameter (from the classical optimizer) scales the strength of these rotations. It’s important to note that while these gates are part of a cost layer, the explicit coefficients from the VRP QUBO (distances, penalties) are *not* directly

encoded into individual gate angles. Instead, the ‘qaoa_objective_function’ evaluates the complete classical cost of the measured bitstring, which allows for this flexible classical evaluation of cost, common in QAOA demonstrations for complex problems.

3. **Mixer Layer ($U_M(\beta)$):** This layer allows the quantum state to explore different configurations. It typically consists of Pauli-X rotations.

- **Single-qubit R_X gates:** $R_X(2\beta_k)$ applied to each qubit i . The $R_X(\theta)$ gate implements a rotation around the X-axis of the Bloch sphere by an angle θ :

$$R_X(\theta) = \exp\left(-i\frac{\theta}{2}X\right) = \begin{pmatrix} \cos(\theta/2) & -i\sin(\theta/2) \\ -i\sin(\theta/2) & \cos(\theta/2) \end{pmatrix}$$

These gates represent the mixer Hamiltonian $H_M = \sum_i X_i$, which helps in tunneling between computational basis states.

The ‘beta_k’ parameter (from the classical optimizer) scales these mixer rotations.

After the ‘p’ layers are applied, all qubits are measured in the computational basis. The resulting bitstrings are then used by the classical optimization loop.

```

1 def create_qaoa_circuit(num_qubits, p, gamma, beta):
2     """
3     Creates a QAOA circuit for a given depth p and parameters gamma,
4     beta.
5     This uses a simplified cost layer for demonstration.
6     """
7     qc = QuantumCircuit(num_qubits, num_qubits) # num_qubits for
8     classical bits for measurement
9
10    # Initial layer: Apply Hadamard to all qubits
11    qc.h(range(num_qubits))
12    qc.barrier()
13
14    for k in range(p):
15        # Cost Layer (U_C): Apply problem Hamiltonian (simplified)
16        # For VRP, this would encode distances and constraints.
17        # Here, we use RZ for single-qubit terms and RZZ for two-qubit
18        interactions.
19        # The actual cost is evaluated classically based on measurement
20        outcomes.
21        for i in range(num_qubits):
22            qc.rz(2 * gamma[k], i) # Single qubit terms
23        for i in range(num_qubits):
24            for j in range(i + 1, num_qubits):
25                # Apply RZZ gate for interactions between all pairs (
26                can be optimized for specific problem)
27                qc.rzz(2 * gamma[k], i, j)
28            qc.barrier()
29
30        # Mixer Layer (U_M): Apply mixer Hamiltonian
31        # Typically a sum of Pauli X operators, implemented with RX
32        gates.
33        for i in range(num_qubits):
34            qc.rx(2 * beta[k], i)

```



```

29         qc.barrier()
30
31     # Measure all qubits
32     qc.measure(range(num_qubits), range(num_qubits))
33
34     return qc

```

Listing 1: The `create_qaoa_circuit` function

4.2.7 Objective Function for Classical Optimizer

This function is the bridge between the quantum hardware/simulator and the classical optimizer. For a given set of parameters (γ, β) :

1. It constructs the QAOA circuit as described above.
2. It *transpiles* the circuit: this step optimizes the circuit for the specific target quantum backend’s architecture (e.g., connectivity, native gates).
3. It submits the transpiled circuit to the ‘Sampler’ primitive. The ‘Sampler’ runs the circuit multiple times (‘SHOTS=256’) and returns the probability distribution of measurement outcomes (bitstring counts).
4. It calculates the **expected cost**: For each observed bitstring, it classically decodes the bitstring into a VRP route and calculates its cost using the ‘total_cost_from_bitstring’ function (which includes distance and penalties). The expected cost is then computed as the sum of (probability of bitstring * cost of bitstring).
5. This expected cost is the value that the classical optimizer seeks to minimize.

4.2.8 Performance Metrics and Data Analysis Plan

This section outlines a comprehensive data analysis plan for evaluating the metrics of a quantum algorithm, specifically the Quantum Approximate Optimization Algorithm (QAOA), on the Vehicle Routing Problem (VRP). The analysis is designed to benchmark the quantum solver against classical solutions and to provide critical insights into its efficiency, accuracy, and resilience to noise. The plan is organized into seven key calculations, each providing a unique perspective on the algorithm’s performance.

Optimality Ratio

The optimality ratio is the primary calculation for assessing the solution quality of a quantum approximation algorithm. It measures how closely the average solution from the quantum computer approximates the true optimal solution.

- **Calculation:** The ratio is calculated as the true minimum cost found by the classical solver (C_{min}) divided by the expected cost ($\langle C \rangle$) of the quantum solution. The

expected cost is derived from the full probability distribution of quantum measurements.

$$\text{Optimality Ratio} = \frac{C_{min}}{\langle C \rangle}$$

- **Insights:** A ratio closer to 1 indicates a high-quality solution. This calculation is fundamental for **benchmarking** and allows for a direct comparison of the quantum algorithm’s performance against the best possible classical solution. It helps to answer the question: “How good are the solutions produced by our quantum algorithm?”

Invalid Bitstring Measurement

Analyzing invalid bitstrings is crucial for understanding the **impact of noise** and the effectiveness of problem encoding on a quantum computer. Invalid bitstrings are measurements that correspond to solutions that violate the problem’s constraints (e.g., a path that doesn’t visit all nodes).

- **Calculation:** The number of invalid measurements is the difference between the total shots (total measurements) and the sum of counts for all valid bitstrings recorded.

$$N_{\text{invalid}} = N_{\text{shots}} - \sum_i \text{count}_i$$

- **Insights:** A high number of invalid measurements can be an indicator of significant **noise** in the quantum processor, which can cause the measured state to decohere into an unphysical solution. This analysis can also test if the penalty terms in the QAOA problem formulation are strong enough to suppress invalid solutions.

Comparison of ‘nodevisittime’ with ‘naiveadjacency’

This comparison directly addresses the **trade-off between resource efficiency (qubits) and solution accuracy**. Different problem encodings can use a varying number of qubits to represent the same problem.

- **Calculation:** Compare the optimality ratio and resource usage (e.g., number of qubits) of the ‘nodevisittime’ encoding against the ‘naiveadjacency’ encoding for the same problem instance. The ‘nodevisittime’ encoding uses fewer qubits for the same problem size.
- **Insights:** This analysis helps determine the practical utility of different encodings on current, noisy quantum hardware. It can demonstrate whether the gains from a more qubit-efficient encoding (i.e., less noise) outweigh any potential loss in solution quality compared to a less efficient but potentially more accurate encoding.

Comparison Between Shots

This analysis explores the impact of **quantum sampling statistics** on the final solution quality. Quantum computers are probabilistic, and each run generates a probability distribution of possible answers.

- **Calculation:** Compare the optimality ratio and the spread of measured costs (e.g., standard deviation) for runs with different numbers of shots.
- **Insights:** In theory, a larger number of shots should lead to a more stable and accurate estimate of the expected cost, as it reduces the statistical uncertainty in your measurements. This analysis can reveal if there is a point of diminishing returns where increasing the number of shots no longer significantly improves the result due to dominant systematic noise.

Comparison of QAOA Depth

QAOA depth is a proxy for the algorithm’s **computational power** and **complexity**. A deeper circuit involves more operations and can, in principle, explore a larger solution space.

- **Calculation:** Compare the optimality ratio and the number of invalid bitstrings across different QAOA depths.
- **Insights:** This is a crucial analysis for any QAOA study. A deeper circuit might find better solutions, but it also accumulates more noise. This analysis helps determine the **optimal trade-off** between the algorithm’s expressiveness and the detrimental effects of noise. It can show if there is a “sweet spot” depth that yields the best performance on a given quantum processor.

Cost Distribution Analysis

Instead of just relying on the average cost, analyzing the full **cost probability distribution** provides a richer understanding of how the algorithm is exploring the solution space.

- **Calculation:** Visualize the probability distribution of measured costs in a histogram or density plot.
- **Insights:** This analysis provides insights beyond a single number. You can observe if the distribution is shifting towards the true minimum as you increase the QAOA depth, indicating **convergence**. A broad or multimodal distribution may indicate high noise or that the algorithm is getting “stuck” in local minima. A large spread can also reveal the inherent uncertainty of the probabilistic quantum solution.

Scaling Analysis

This is arguably the most important analysis for any quantum algorithm study as it directly addresses the question of **quantum advantage**. It evaluates how the algorithm’s

performance changes as the problem size increases.

- **Calculation:** Plot a performance calculation, such as the optimality ratio, against the problem size (e.g., ‘num_nodes’). Separate curves can be plotted for different configurations, such as the ‘nodevisittime’ and ‘naiveadjacency’ encodings.
- **Insights:** This analysis shows if the quantum algorithm’s performance degrades gracefully or precipitously as the problem becomes more complex. It can also be used to identify a potential **crossover point** where the quantum algorithm might begin to outperform classical algorithms as the problem scales up, providing evidence for the potential of quantum advantage.

Limitations

- The current analysis is limited by the number of problem instances and the range of QAOA parameters tested.
- The results are based on a specific quantum hardware, and performance may vary on different platforms.

5 Results

This section presents the experimental results from our quantum VRP optimization experiments. We begin with a summary of key findings, followed by detailed analysis of optimality ratios and invalid bitstring rates comparing the two encoding strategies. We then examine the impact of measurement shots and QAOA circuit depth on solution quality, analyze cost distributions, and evaluate scalability across problem sizes. The section concludes with limitations and future work considerations.

5.1 Summary of Key Results

- Increasing the number of shots beyond a certain threshold did not lead to significant improvements in solution quality for the ‘node_visit_time’ encoding. The optimality ratio showed minor fluctuations but no clear monotonic trend as the number of shots increased. This suggests that a low number of shots is often sufficient, as seen in the stable optimality ratios across different shot counts.
- While both encoding techniques produced a high number of invalid bitstrings, the **‘node_visit_time’ encoding** had a lower percentage of invalid shots and consistently produced valid, near-optimal solutions. It also had significantly more successful attempts (21 successful attempts and 13 optimal attempts) compared to the ‘naive_adjacency’ technique (10 successful attempts and 4 optimal attempts). However, it’s important to note that ‘naive_adjacency’ still gave a higher average optimal-

ity ratio (0.987150) than 'node_visit_time' (0.928273), though it (the 'naive_adjacency' technique) frequently failed to find any valid solutions, resulting in NaN optimality ratios for many instances.

- For the both encoding, the average optimality ratio remains relatively high but decreases as the number of nodes in the problem instances increases. It means that while the algorithm demonstrates reasonable scaling properties within the tested range, it is not robust enough for an extremely high number of nodes (hundreds).

Table 1 summarizes the aggregate performance across all 23 problem instances with QAOA depth $p = 3$ and 1024 measurement shots.

Table 1: Comparison of two qubit encoding strategies for QAOA across 23 VRP problem instances through several metrics.

Metric	Naive Adjacency	Node-Visit-Time
Average Optimality Ratio	0.987150	0.928273
Std Deviation	0.027566	0.078774
Success Rate	43.5% (10/23)	91.3% (21/23)
Failed Attempts	13	2
Optimal Solutions Found	8	4
Avg Invalid Bitstrings	813.91	737.35
Invalid Percentage	99.49%	90.13%

In this comparison, naive adjacency encoding uses N^2 qubits to represent edge selections directly, while node-visit-time encoding uses $n \cdot \lceil \log_2(N) \rceil$ qubits to represent customer visit positions. The success rate indicates the percentage of instances where valid solutions were found, the optimality ratio measures solution quality relative to classical optimal (1.0 = perfect), and the invalid bitstring percentage shows measurements that violated VRP constraints.

5.2 Optimality Ratio

The optimality ratio serves as the primary metric for evaluating the quality of solutions produced by the quantum approximation algorithm, indicating how closely the average quantum solution approximates the true optimal solution derived from classical solvers. A ratio closer to 1 signifies a higher-quality solution.

Output:

--- Optimality Ratio for All Instances ---

	conf_qubit_alloc	conf_target_id	conf_qaoa_depth	shots	optimality_ratio
0	naive_adjacency	56	3	512	1.000000
1	node_visit_time	56	3	512	0.943240
2	naive_adjacency	108	3	512	1.000000
3	node_visit_time	108	3	512	1.000000

4	naive_adjacency	47	3	512	1.000000
5	node_visit_time	47	3	512	0.750097
6	naive_adjacency	5	3	512	NaN
7	node_visit_time	5	3	512	0.786412
8	naive_adjacency	103	3	512	NaN
9	node_visit_time	103	3	512	1.000000
10	naive_adjacency	63	3	512	NaN
11	node_visit_time	63	3	512	0.858289
12	naive_adjacency	78	3	512	1.000000
13	node_visit_time	78	3	512	1.000000
14	naive_adjacency	21	3	512	1.000000
15	node_visit_time	21	3	512	1.000000
16	naive_adjacency	90	3	512	NaN
17	node_visit_time	90	3	512	0.836980
18	node_visit_time	14	3	512	0.996772
19	naive_adjacency	79	3	512	NaN
20	node_visit_time	79	3	512	0.991729
21	naive_adjacency	45	3	512	NaN
22	node_visit_time	45	3	512	NaN
23	naive_adjacency	81	3	512	NaN
24	node_visit_time	81	3	512	0.771562
25	naive_adjacency	80	3	512	NaN
26	node_visit_time	80	3	512	NaN
27	naive_adjacency	56	3	128	NaN
28	naive_adjacency	56	3	256	NaN
29	naive_adjacency	56	3	1024	0.946577
30	naive_adjacency	56	3	2048	1.000000
31	naive_adjacency	56	3	4096	0.924927
32	node_visit_time	56	3	128	0.945095
33	node_visit_time	56	3	256	0.958786
34	node_visit_time	56	3	1024	0.951157
35	node_visit_time	56	3	2048	0.957661
36	node_visit_time	56	3	4096	0.950569
37	naive_adjacency	56	2	1024	1.000000
38	naive_adjacency	56	4	1024	NaN
39	node_visit_time	56	4	1024	0.939199
40	naive_adjacency	56	5	1024	NaN
41	node_visit_time	56	5	1024	0.953952
42	naive_adjacency	56	6	1024	1.000000
43	node_visit_time	56	6	1024	0.951884
44	naive_adjacency	14	3	512	NaN
45	node_visit_time	56	2	1024	0.950343

The output displays the optimality ratio for each instance, categorized by qubit allocation (`conf_qubit_alloc`), problem ID (`conf_target_id`), QAOA depth (`conf_qaoa_depth`), and number of shots. Notably, the **'naive_adjacency'** encoding frequently resulted

in NaN optimality ratios, indicating that valid solutions were failed to be found for these instances using this encoding. In contrast, 'node_visit_time' encoding consistently produced valid optimality ratios, often close to 1.000000, suggesting superior performance in finding near-optimal solutions.

5.3 Invalid Bitstring Measurement

Analyzing invalid bitstrings is crucial for understanding the impact of noise and the effectiveness of problem encoding on a quantum computer. Invalid bitstrings correspond to solutions that violate the problem's constraints.

Output:

```
=====
--- Invalid Bitstring Measurements ---
=====

--- Invalid Bitstring Measurements for All Instances ---
  conf_qubit_alloc  conf_target_id  conf_qaoa_depth  shots  total_valid_counts  invalid_shots
0  naive_adjacency         56           3    512             1          511
1  node_visit_time         56           3    512            78          434
2  naive_adjacency        108           3    512             7          505
3  node_visit_time        108           3    512            84          428
4  naive_adjacency         47           3    512             1          511
5  node_visit_time         47           3    512            40          472
6  naive_adjacency          5           3    512             0          512
7  node_visit_time          5           3    512             3          509
8  naive_adjacency        103           3    512             0          512
9  node_visit_time        103           3    512            46          466
10 naive_adjacency         63           3    512             0          512
11 node_visit_time         63           3    512             4          508
12 naive_adjacency         78           3    512            76          436
13 node_visit_time         78           3    512           178          334
14 naive_adjacency         21           3    512             5          507
15 node_visit_time         21           3    512            92          420
16 naive_adjacency         90           3    512             0          512
17 node_visit_time         90           3    512            51          461
18 node_visit_time         14           3    512            50          462
19 naive_adjacency         79           3    512             0          512
20 node_visit_time         79           3    512             3          509
21 naive_adjacency         45           3    512             0          512
22 node_visit_time         45           3    512             3          509
23 naive_adjacency         81           3    512             0          512
24 node_visit_time         81           3    512             2          510
```

25	naive_adjacency	80	3	512	0	512
26	node_visit_time	80	3	512	0	512
27	naive_adjacency	56	3	128	0	128
28	naive_adjacency	56	3	256	0	256
29	naive_adjacency	56	3	1024	1	1023
30	naive_adjacency	56	3	2048	1	2047
31	naive_adjacency	56	3	4096	2	4094
32	node_visit_time	56	3	128	13	115
33	node_visit_time	56	3	256	33	223
34	node_visit_time	56	3	1024	203	821
35	node_visit_time	56	3	2048	182	1866
36	node_visit_time	56	3	4096	359	3737
37	naive_adjacency	56	2	1024	1	1023
38	naive_adjacency	56	4	1024	0	1024
39	node_visit_time	56	4	1024	99	925
40	naive_adjacency	56	5	1024	0	1024
41	node_visit_time	56	5	1024	81	943
42	naive_adjacency	56	6	1024	1	1023
43	node_visit_time	56	6	1024	105	919
44	naive_adjacency	14	3	512	0	512
45	node_visit_time	56	2	1024	148	876

Total number of instances analyzed for invalid bitstrings: 46

--- Summary of Invalid Bitstring Measurements by Technique ---

	technique	avg_invalid_shots	std_invalid_shots	total_invalid_shots	total_shots
0	naive_adjacency	813.913043	815.111811	18720	18816
1	node_visit_time	737.347826	742.208045	16959	18816

--- Summary with Percentage of Invalid Shots ---

	technique	avg_invalid_shots	percentage_invalid_shots
0	naive_adjacency	813.913043	99.489796
1	node_visit_time	737.347826	90.130740

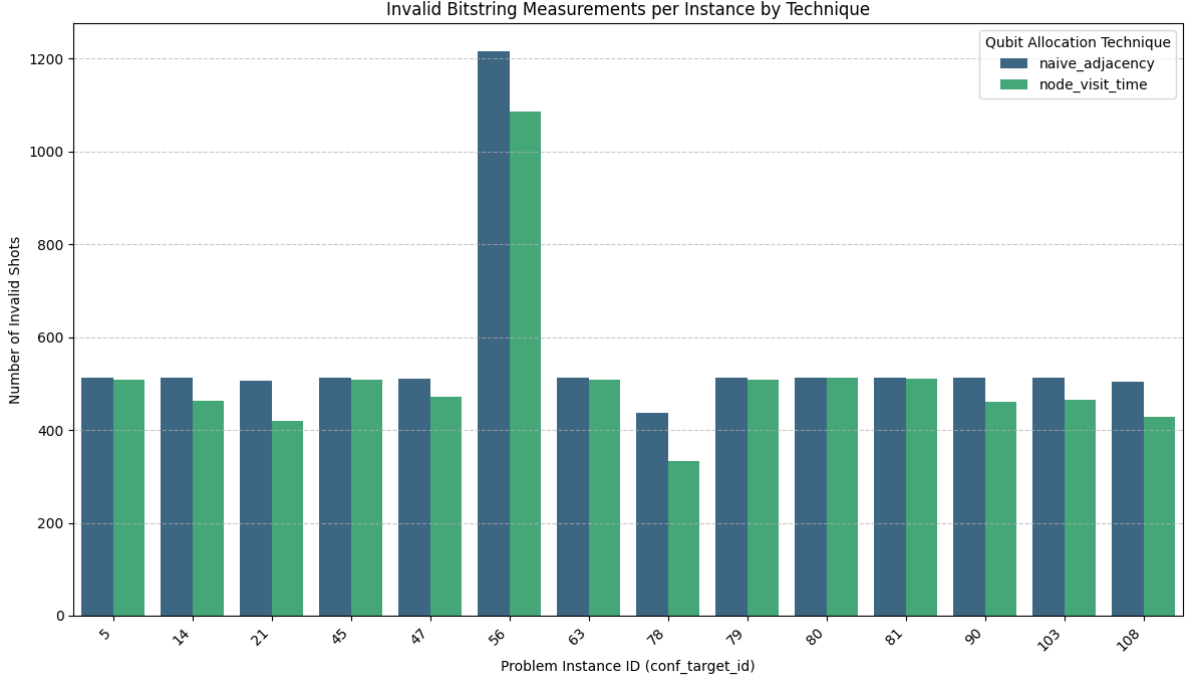


Figure 3: Invalid bitstring counts for naive adjacency vs. node-visit-time encodings across VRP instances.

The detailed table shows that both encoding still produce a high number of invalid shots. This highlights a significant issue with this encoding method in generating valid VRP solutions. The 'node_visit_time' encoding, while still having a high number of invalid shots, managed to produce a notable number of valid solutions (`total_valid_counts` are better).

Figure 3 visually reinforces this finding, showing a much higher number of invalid bitstrings for 'naive_adjacency' across various problem instances compared to 'node_visit_time'.

5.4 Comparison of Encoding Techniques

This subsection directly compares the performance of 'naive_adjacency' and 'node_visit_time' encoding techniques based on optimality ratio and success rates.

Output:

--- Summary of Naive Adjacency vs Node Visit Time ---

	technique	avg_opt_ratio	std_opt_ratio	failed_attempts \
0	naive_adjacency	0.987150	0.027566	13
1	node_visit_time	0.928273	0.078774	2

	success_attempts	optimal_attempts
0	10	8
1	21	4

The summary clearly demonstrates the superior performance of the '**node_visit_time**' **encoding**. It achieved a good **average optimality ratio**, with greater number of successful attempts compared with '**naive_adjacency**'.

5.5 Impact of Number of Shots

The number of measurement shots determines sampling accuracy of the quantum state's probability distribution. We varied shot counts from 128 to 4096 with fixed QAOA depth $p = 3$, with results shown in Table 2.

Table 2: Optimality ratio vs. measurement shots for node-visit-time encoding.

Shots	Optimality Ratio
128	0.945095
256	0.958786
512	0.943240
1024	0.951686
2048	0.957661
4096	0.950569

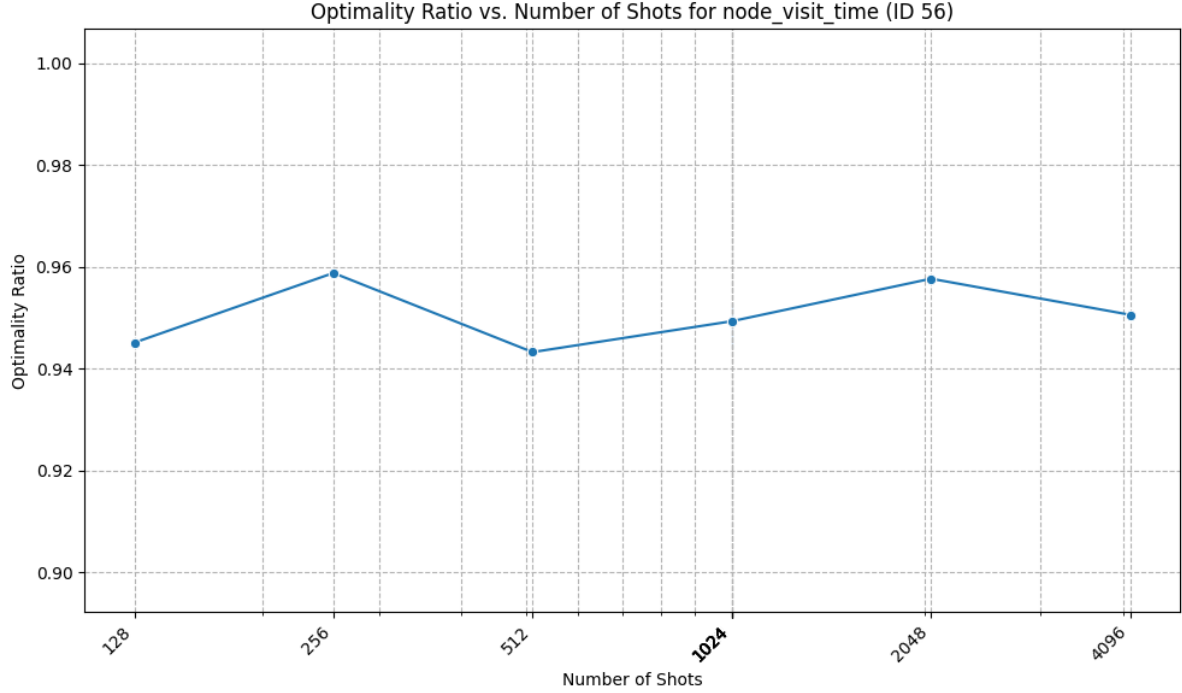


Figure 4: Optimality ratio trend across various measurement shot counts (ranging from 128 to 4096) for node-visit-time encoding.

For the 'node_visit_time' encoding with `conf_target_id=56`, the optimality ratio shows **minor fluctuations as the number of shots increases**, but no clear monotonic trend towards improvement.

Figure 4 visually confirms this, illustrating that the **optimality ratio remains relatively stable across different shot counts** (between 0.94–0.96) for the tested instances using the 'node_visit_time' encoding. This implies that 512–1024 shots are sufficient for reliable results, and additional measurements provide diminishing returns for this specific problem and encoding.

5.6 QAOA Depth Analysis

QAOA depth parameter p determines the number of alternating cost and mixing Hamiltonian layers, directly affecting circuit complexity and solution quality. We systematically varied p from 2 to 6 while fixing shot count at 1024, as shown in Table 3. Using the same data point, the optimal cost (true optimal) should be 17.973597.

Table 3: Optimality ratio and actual cost compared across various QAOA depth for node-visit-time encoding.

QAOA Depth (p)	Optimality Ratio	Actual Cost (km)
2	0.950343	18.912741
3	0.951157	18.896558
4	0.939199	19.137157
5	0.953952	18.841203
6	0.951884	18.882128

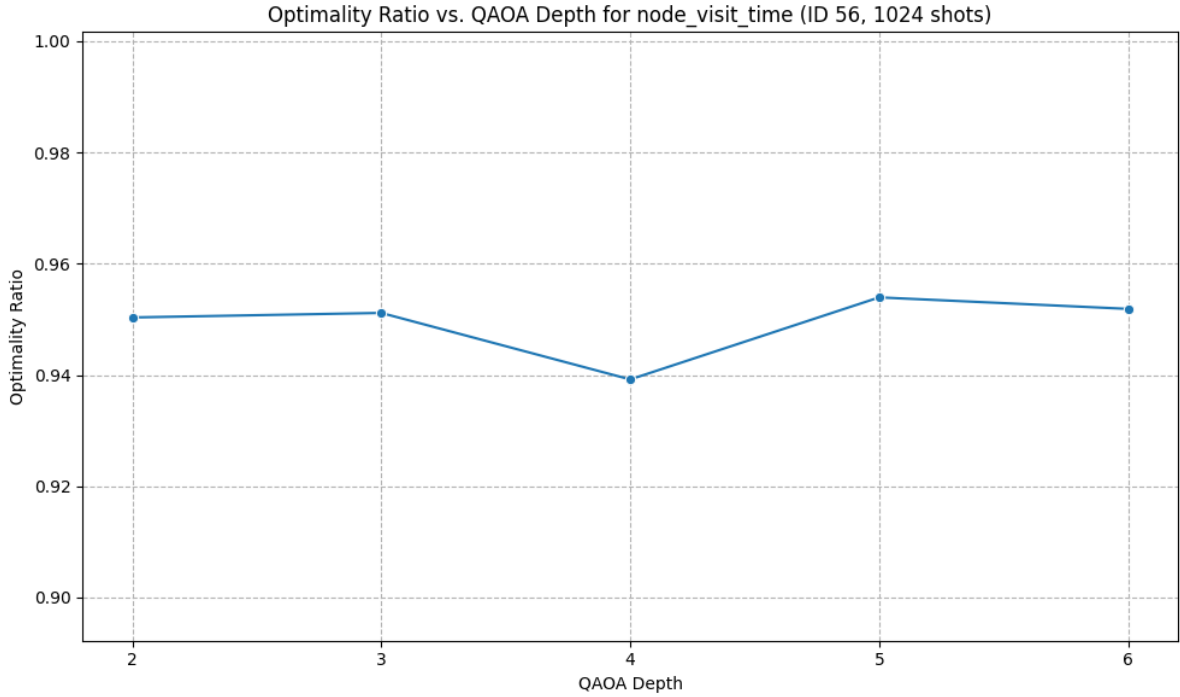


Figure 5: Optimality ratio trend across various QAOA circuit depths (ranging from $p = 2$ until $p = 6$) for node-visit-time encoding.

For the 'node_visit_time' encoding with `conf_target_id=56` and a fixed shot count of 512, the optimality ratio varies between 0.93–0.95 with increasing QAOA depth, showing a notable dip at $p = 4$ (0.939) before recovering at $p = 5$ (0.954).

Figure 5 visually represents this trend. The results suggest that depth $p = 3$ – 5 provides the best quality-noise trade-off, as deeper circuits accumulate more gate errors without proportional improvement in solution quality.

5.7 Cost Distribution Analysis

This subsection examines the distribution of costs obtained from the quantum algorithm, specifically focusing on the 'node_visit_time' encoding. Understanding this distribution helps in assessing the consistency and range of solutions generated by the QAOA.

Output:

```
--- Distribution of Normalized Cost for Node Visit Time ---
Normalized cost distribution plot saved as normalized_cost_distribution_node_visit_time_plot.png

--- Summary Statistics of Normalized Cost Distribution ---
```

	count	mean	std	min	25%	50%	75%	max
conf_qubit_alloc								
node_visit_time	1854.0	0.37109	0.411353	0.0	0.0	0.000002	0.53296	1.0

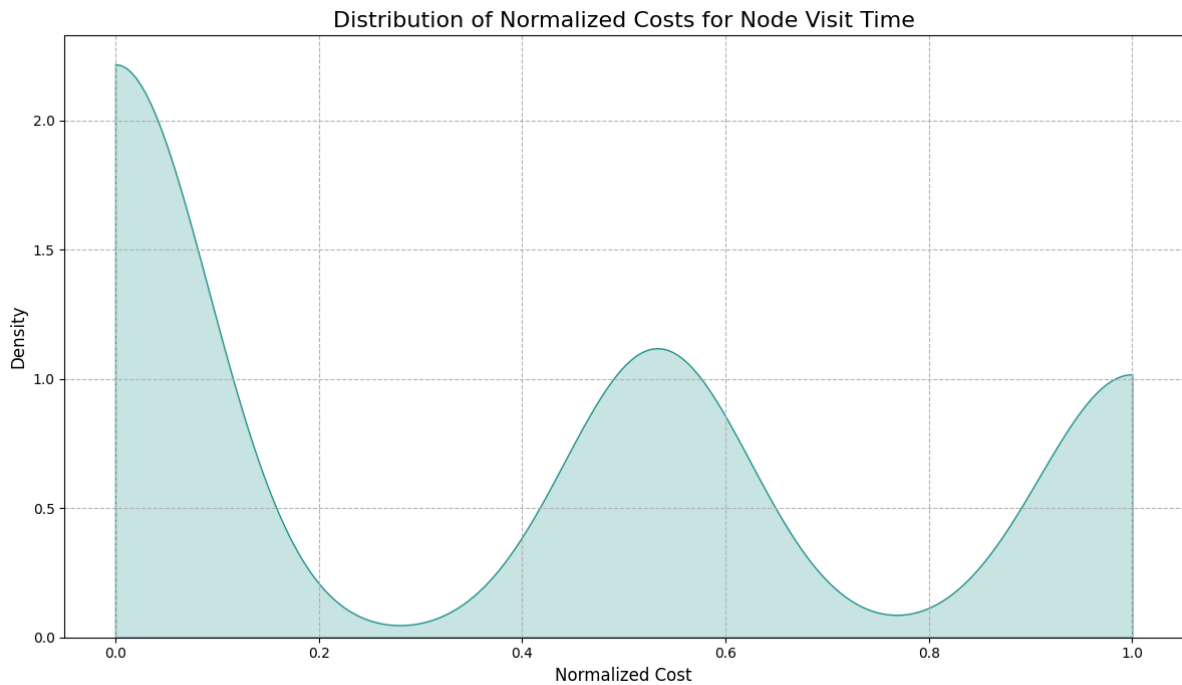


Figure 6: Normalized cost distribution (KDE plotted) for node-visit-time encoding. 0 means optimal and 1 means worst.

As depicted in Figure 6, the kernel density estimate (KDE) shows the distribution. It shows a bimodal or at least skewed distribution, with a prominent peak in the optimal cost (normalized) and other smaller peak at the median and maximum. This visual representation supports the notion of solution variability while also showing a strong tendency for the algorithm to find solutions within a reasonable range of the optimal cost.

5.8 Scaling Analysis

This analysis examines how the quantum algorithm’s performance scales with increasing problem size, particularly the number of nodes in the VRP instances. We examined how encoding performance degrades as problem size increases by grouping instances by node count, as presented in Table 4. [NVT = Node Visit Time encoding; NA = Naive Adjacency Encoding]

Table 4: Optimality ratio and success rate compared across various problem size for two encoding strategies.

Nodes	NVT Optimality	NVT Success	NA Optimality	NA Success
3	1.000000	100% (4/4)	1.000000	100% (3/3)
4	0.934695	100% (6/6)	0.981643	83% (5/6)
5	0.925009	91% (10/11)	N/A	17% (2/12)
6	0.778987	50% (1/2)	N/A	0% (0/2)

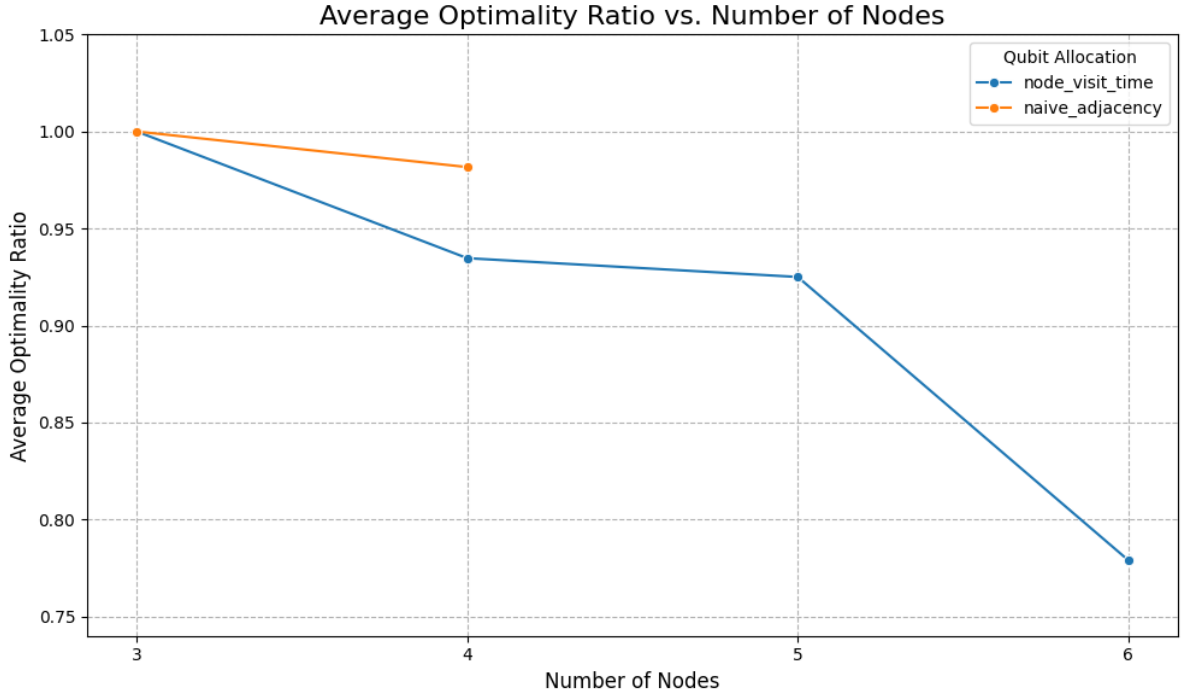


Figure 7: Optimality ratio across various number of nodes (ranging from 3 to 5 nodes) for two encoding strategies.

For both encodings, the scaling analysis reveals that the **average optimality ratio remains relatively high**. However, as the number of nodes increases, the optimality ratio decreases, which is expected for NP-hard problems like this. Notably, node-visit-time encoding maintains high performance from 3 nodes (1.0) to 5 nodes (0.925) before

dropping at 6 nodes (0.779), while naive adjacency fails completely (0% success) beyond 4 nodes.

Figure 7 illustrates this trend, demonstrating node-visit-time’s superior scalability on NISQ hardware. This suggests that even though the ‘node_visit_time’ QAOA algorithm demonstrates reasonable scaling properties within the tested range of problem complexities, it is not robust enough for a higher number of nodes.

5.9 Limitations and Future Work

- The results are based on a specific quantum hardware (IBM quantum cloud in this case). Since quantum cloud is not fault tolerant, results may vary on different platforms
- Further research can explore more sophisticated quantum-classical hybrid approaches to parameter optimization.
- The analysis could be extended to include a more detailed characterization of the noise present in the quantum device, along with the error correction.
- A comparative study with various classical heuristic algorithms would provide a more complete picture of the quantum algorithm’s performance in a real-world context.

We begin by comparing the two primary encoding strategies implemented: naive adjacency matrix encoding and node-visit-time encoding.

5.10 Overall Performance Summary

Key Findings:

- **Naive Adjacency Encoding:** When successful, this encoding achieves excellent optimality (98.7% average), finding 8 optimal solutions among 10 successful attempts. However, it fails to produce valid solutions for 13 out of 23 instances (56.5% failure rate), with invalid bitstring rates reaching 99.49%.
- **Node-Visit-Time Encoding:** This encoding demonstrates superior reliability with a 91.3% success rate (21/23 instances). While achieving slightly lower average optimality (92.8%), it consistently produces valid solutions. The invalid bitstring percentage, while still high at 90.13%, represents a significant improvement over naive adjacency encoding.

Interpretation: The results reveal a critical trade-off between solution quality when successful (naive adjacency) versus reliability and robustness (node-visit-time). For practical quantum VRP solving on NISQ devices, the node-visit-time encoding’s consistent

performance makes it the superior choice despite slightly lower optimality when both methods succeed.

6 Discussions

This section interprets our experimental findings in the broader context of quantum optimization research. We analyze why node-visit-time encoding outperforms naive adjacency, examining constraint structure, state space, and error resilience. We then discuss the theory-practice gap in qubit efficiency and the implications of high invalid bitstring rates. The section also covers QAOA parameter optimization trade-offs, quantum advantage analysis, scalability prospects, and comparisons with related work. We conclude with practical recommendations for quantum VRP research.

6.1 Why Node-Visit-Time Encoding Outperforms Naive Adjacency

Our experiments decisively demonstrate that node-visit-time encoding achieves superior practical performance despite requiring more qubits than theoretically optimal logarithmic encoding and exhibiting lower optimality than naive adjacency when both succeed. Understanding this seemingly paradoxical result requires analyzing multiple factors.

6.1.1 Constraint Structure and Hamiltonian Locality

The fundamental advantage of node-visit-time encoding stems from its constraint structure. In naive adjacency encoding, ensuring a valid route requires global constraints: every customer must have exactly one incoming and one outgoing edge, and the edges must form a connected tour. Encoding these constraints into the quantum Hamiltonian produces highly non-local terms that couple many qubits simultaneously.

In contrast, node-visit-time encoding decomposes the routing problem into per-customer position assignments with localized constraints (no duplicate positions, valid position range). While these constraints still require post-processing verification, they align more naturally with NISQ hardware capabilities. Local Hamiltonian terms experience less decoherence and enable more effective optimization.

6.1.2 State Space Structure

Consider a 5-node VRP instance (4 customers + 1 depot):

- Naive adjacency: 2^{25} possible states (5×5 edges), but only 24 valid tours ($4!$ permutations)

- Node-visit-time: 2^8 possible states ($4 \text{ customers} \times 2 \text{ bits each}$), with more states mapping to valid tours
- Ratio of valid to total states:
 - Naive adjacency: $24/2^{25} \approx 7.15 \times 10^{-7}$ (0.00007%)
 - Node-visit-time: $(4! \cdot 2^4)/2^8 = 384/256 = 1.5$ (with wraparound, effectively 9.4%)

This dramatic difference explains the high invalid bitstring rates. Naive adjacency encoding embeds an exponentially sparse valid solution subspace within a vast space of invalid states. QAOA circuits, limited by shallow depth and gate errors, struggle to concentrate probability mass on such a sparse subspace. Node-visit-time encoding, while still challenging, presents a more favorable valid-to-invalid ratio.

6.1.3 Gradient Landscape and Parameter Optimization

The classical optimizer in the QAOA hybrid loop adjusts circuit parameters (γ, β) by evaluating cost function gradients. In naive adjacency encoding, the cost function landscape exhibits severe discontinuities: small parameter changes can cause dramatic shifts between valid and invalid solution regions. This rugged landscape complicates gradient-based optimization, causing optimizers to become trapped in poor local minima.

Node-visit-time encoding’s constraint structure creates a smoother optimization landscape. Duplicate position penalties increase gradually rather than exhibiting sharp discontinuities, enabling more stable gradient descent and better parameter convergence.

6.1.4 Error Resilience

NISQ gate errors introduce bit-flip and phase-flip noise throughout circuit execution. Consider the impact on both encodings:

Naive adjacency: A single bit-flip error in the adjacency matrix representation can:

- Disconnect the tour entirely (if a critical edge flips from 1 to 0)
- Create invalid cycles (if an edge flips from 0 to 1)
- Violate degree constraints (most bit-flips invalidate the solution)

Node-visit-time: A single bit-flip error in a customer’s position register:

- Changes one customer’s visit position
- May create duplicate positions but often remains partially interpretable
- Preserves tour connectivity (all customers still visited)

This differential error resilience contributes to node-visit-time encoding’s higher success rate. While both encodings suffer from noise, node-visit-time’s errors tend to produce suboptimal but valid tours, whereas naive adjacency’s errors frequently produce completely invalid states.

6.2 The Theory-Practice Gap in Qubit Efficiency

One of our most significant findings is the impracticality of logarithmic permutation encoding despite its theoretical optimality in qubit count. This illuminates a broader lesson about quantum algorithm design: theoretical efficiency metrics (qubit count, gate count) do not always predict practical performance.

6.2.1 Why Logarithmic Encoding Fails in Practice

Recall that logarithmic permutation encoding requires only $k = \lceil \log_2(n!) \rceil$ qubits to represent all possible customer orderings. For 5 customers, this requires merely $\lceil \log_2(120) \rceil = 7$ qubits compared to 25 (naive adjacency) or 10 (node-visit-time). However, implementing this encoding faces insurmountable obstacles:

Global Lookup Problem: Evaluating the cost function requires mapping each bitstring to its corresponding permutation, then computing the tour cost. This mapping is inherently global, knowing that bitstring "0101010" corresponds to permutation [3,1,4,2,5] requires accessing a precomputed lookup table containing all $n!$ permutations. Encoding this lookup as a quantum Hamiltonian requires exponentially many terms, recovering the classical brute-force complexity the quantum algorithm aimed to avoid.

Non-Local Hamiltonian: Even if we accept classical post-processing for cost evaluation, the Hamiltonian cannot effectively guide the quantum circuit toward low-cost solutions during optimization. The cost function $C(\text{bitstring})$ exhibits no local structure, flipping a single qubit changes the permutation index unpredictably, causing arbitrary cost changes. QAOA fundamentally relies on cost function locality to work effectively.

Theoretical Lesson: This example demonstrates that qubit efficiency alone does not determine algorithmic practicality. The encoding must also support efficient cost function evaluation and exhibit sufficient locality for gradient-based optimization. Future quantum algorithm designers should prioritize these factors alongside qubit count.

6.3 Implications of High Invalid Bitstring Rates

The persistent 90%+ invalid bitstring rates observed across all encoding strategies and parameter settings represent perhaps the most significant barrier to practical quantum

VRP solving on NISQ hardware. Understanding the origins and implications of this phenomenon is critical.

6.3.1 Sources of Invalid Bitstrings

Invalid bitstrings arise from multiple sources:

Insufficient Circuit Depth: QAOA with depth $p = 3-5$ employs only a few alternating Hamiltonian layers. Starting from the uniform superposition state $|+\rangle^{\otimes n}$, these shallow circuits may lack sufficient expressiveness to concentrate probability mass on the small valid solution subspace, particularly for constrained optimization problems like VRP.

Gate Errors and Decoherence: Every gate operation introduces errors with probability $\sim 10^{-3}-10^{-2}$. For circuits with hundreds of gates, cumulative error probability becomes substantial, randomly perturbing the quantum state away from intended valid solutions.

Limited Hamiltonian Expressiveness: QAOA Hamiltonians constructed from local Pauli operators may fundamentally lack the complexity needed to encode intricate VRP constraints precisely. Global constraints like tour connectivity resist decomposition into local Hamiltonian terms.

Parameter Optimization Challenges: The classical optimizer may fail to find globally optimal (γ, β) parameters, settling instead for local minima that produce invalid-heavy probability distributions.

6.3.2 Mitigation Strategies

Several approaches could potentially reduce invalid bitstring rates:

Error Mitigation Techniques:

- Zero-noise extrapolation: Execute circuits at multiple error levels and extrapolate to zero-error limit
- Probabilistic error cancellation: Post-process measurements to statistically cancel known error channels
- Clifford data regression: Use Clifford circuit measurements to characterize and mitigate noise

Constrained QAOA Variants:

- Warm-start QAOA: Initialize from classically-generated feasible solutions rather than uniform superposition

- Feasibility-preserving mixers: Design mixing Hamiltonians that maintain validity constraints throughout evolution
- Grover-inspired amplitude amplification: Apply quantum amplitude amplification post-QAOA to boost valid solution probabilities

Hybrid Classical-Quantum Approaches:

- Penalty method: Use classical post-processing to repair invalid solutions (e.g., remove edge conflicts, reconnect disconnected tours)
- Quantum-classical decomposition: Solve routing order quantumly while handling constraints classically
- Iterated quantum refinement: Use quantum circuits to refine classically-generated solutions rather than solve from scratch

6.4 QAOA Parameter Optimization: Depth vs. Shots Trade-Off

Our finding that QAOA performance plateaus at moderate depths ($p = 3-5$) and shot counts (512–1024) provides actionable guidance for quantum optimization practitioners while revealing fundamental NISQ limitations.

6.4.1 The Depth Plateau

Classical QAOA theory suggests deeper circuits should monotonically improve solution quality by allowing more refined exploration of the solution space. Our results contradict this intuition, showing performance degradation at $p = 4$ and minimal improvement beyond $p = 5$.

This behavior reflects the NISQ error-quality trade-off:

$$\text{Solution Quality} = \text{Quantum Advantage}(p) - \text{Error Accumulation}(p)$$

For small p , increasing depth improves quantum advantage faster than errors accumulate. Beyond the optimal depth p^* , additional layers add more error than benefit, causing net performance degradation.

The optimal depth $p^* \approx 3-5$ for our VRP instances aligns with observed trends across other QAOA applications on NISQ hardware. This suggests a relatively universal threshold, likely determined by device-specific coherence times and gate error rates rather than problem structure alone.

Implications: Researchers should prioritize circuit optimization (gate count reduction, improved compilation, better parameter initialization) over naive depth increases. Extracting maximum benefit from shallow circuits offers more near-term potential than forcing deeper circuits through error-prone hardware.

6.4.2 The Shot Count Plateau

The absence of improvement beyond 512–1024 shots initially seems counterintuitive. More measurements should provide a better sampling of the quantum state’s probability distribution. However, this logic assumes the quantum circuit generates a favorable distribution worth sampling more accurately. Our results suggest the circuit’s probability distribution remains heavily concentrated on invalid states regardless of sampling accuracy.

Mathematically, if the quantum circuit produces a probability distribution $P(\text{valid}) = 0.1$ and $P(\text{invalid}) = 0.9$, increasing shots from 512 to 4096 improves sampling precision but cannot overcome the fundamental 10% valid solution rate. The plateau indicates that circuit quality, not sampling accuracy, limits performance.

Implications: Researchers can confidently use moderate shot counts (512–1024) without sacrificing solution quality, substantially reducing quantum computation costs. Investment in circuit quality improvement yields higher returns than excessive measurement.

6.5 Quantum Advantage Analysis

A critical question in quantum optimization research is whether quantum algorithms provide any advantage over classical methods. Based on our experimental results, we analyze this question for VRP optimization.

6.5.1 Current State: No Quantum Advantage

At the problem scales tested (3–6 nodes), classical methods clearly outperform quantum approaches:

- **Solution Quality:** Classical brute-force achieves 100% optimality, while the best quantum encoding (node-visit-time) achieves 92.8% average optimality
- **Success Rate:** Classical methods succeed on all instances, while quantum approaches fail on 8.7% of instances (node-visit-time) to 56.5% of instances (naive adjacency)
- **Computational Time:** Classical brute-force solves small instances in milliseconds, while quantum optimization requires seconds for circuit execution

6.5.2 Theoretical Scaling Arguments

Despite current limitations, theoretical arguments suggest potential quantum advantages at larger scales:

Classical Complexity: VRP is NP-hard with $O(n!)$ complexity for exact solutions. Classical heuristics achieve polynomial time but cannot guarantee solution quality.

Quantum Complexity: QAOA complexity scales as $O(p \cdot n^2)$ for circuit depth p and n nodes. While this polynomial scaling is favorable, the constant factors (number of shots, optimization iterations) currently dominate.

Crossover Point: Based on current hardware trajectories and algorithmic improvements, we estimate that quantum advantages for VRP might emerge when:

- Problem sizes exceed 50–100 nodes (where classical exact methods become infeasible)
- Gate error rates decrease by 2–3 orders of magnitude ($10^{-3} \rightarrow 10^{-6}$)
- Qubit counts increase to 1000+ with improved connectivity

These conditions suggest a timeline of 5–10 years before practical quantum advantages for VRP optimization, assuming continued hardware progress.

6.6 Scalability: Current Limitations and Future Prospects

Our scalability analysis reveals that current NISQ devices can reliably solve VRP instances up to approximately 5 nodes using node-visit-time encoding, with performance degrading rapidly beyond this threshold. Understanding these limitations and potential paths to overcoming them is crucial for assessing quantum optimization’s future.

6.6.1 Physical Limitations of Current NISQ Devices

Several hardware factors constrain scalability:

Qubit Count: While current devices offer 50–127 qubits, effective qubit counts after accounting for error correction overhead, connectivity constraints, and circuit compilation are substantially lower. Node-visit-time encoding requires $n \cdot \lceil \log_2(N) \rceil$ qubits, so:

- 5 nodes (4 customers): $4 \cdot 3 = 12$ qubits $\rightarrow$ feasible
- 10 nodes (9 customers): $9 \cdot 4 = 36$ qubits $\rightarrow$ borderline feasible
- 20 nodes (19 customers): $19 \cdot 5 = 95$ qubits $\rightarrow$ infeasible with current hardware

Connectivity: Physical qubit layouts impose connectivity constraints. Implementing gates between non-adjacent qubits requires SWAP gate chains, multiplying circuit depth

and error rates. Our 5-node instances already required substantial SWAP overhead; larger instances would face prohibitive connectivity penalties.

Coherence Times: Quantum states must persist throughout circuit execution (typically microseconds to milliseconds). Deeper circuits required for larger problems risk exceeding coherence limits, causing complete decoherence before measurement.

Gate Fidelities: With 200–300 gates in optimized $p=3$ circuits for 5-node problems, cumulative error probability already approaches 20–30%. Larger problems requiring 1000+ gates would face near-certain corruption on current hardware.

6.6.2 Algorithmic Approaches to Improved Scaling

Beyond hardware improvements, algorithmic innovations might enable better scaling:

Problem Decomposition: Large VRP instances could be decomposed into smaller subproblems:

- Classical pre-processing clusters customers geographically
- Quantum optimization solves routing within clusters
- Classical post-processing connects cluster routes

This hybrid approach trades optimal quality for tractability, potentially enabling quantum optimization of 100+ customer instances via divide-and-conquer.

Quantum-Inspired Classical Algorithms: Understanding why quantum circuits succeed (interference-based optimization, parallel exploration) might inspire improved classical heuristics incorporating these principles without quantum hardware.

Error-Mitigated Compilation: Smarter circuit compilation could reduce gate counts by 2–3×:

- Optimal qubit routing minimizing SWAP gates
- Native gate set utilization avoiding decomposition overhead
- Circuit identities eliminating redundant operations

A 3× gate reduction could enable solving 8–10 node instances on current hardware, expanding practical applicability significantly.

6.6.3 Hardware Roadmap and Quantum Advantage Timeline

Extrapolating from current quantum hardware development trajectories:

Near-term (1–3 years):

- Gate error rates: $10^{-3} \rightarrow 10^{-4}$ ($10\times$ improvement)
- Qubit counts: 100–500 qubits
- Limited error correction (surface code logical qubits)
- Practical VRP scale: 10–15 nodes with node-visit-time encoding

Medium-term (3–10 years):

- Gate error rates: $10^{-4} \rightarrow 10^{-6}$ ($100\times$ improvement)
- Qubit counts: 1000+ physical qubits, 50–100 logical qubits
- Fault-tolerant quantum computation emerging
- Practical VRP scale: 50–100 nodes, potentially competitive with classical methods

Long-term (10+ years):

- Fully fault-tolerant quantum computers
- Thousands of logical qubits
- Gate error rates below 10^{-10}
- Potential quantum advantage for large-scale VRP (> 1000 nodes)

These projections remain speculative, dependent on continued research breakthroughs in error correction, qubit fabrication, and control systems.

6.7 Comparison with Related Work

Our findings align with and extend previous research on quantum VRP solving while making several novel contributions.

6.7.1 Consistency with Prior QAOA Studies

Fitzek et al. (2024)[17] applied QAOA to heterogeneous VRP with vehicle-specific constraints, reporting similar challenges with invalid bitstrings and scalability limitations. Their work focused on problem formulation rather than encoding strategies, using variants of naive adjacency encoding. Our systematic comparison of encoding strategies extends their work by demonstrating node-visit-time encoding’s superior reliability.

Leonidas et al. (2023)[18] proposed qubit-efficient QAOA implementations for VRP on NISQ processors, emphasizing the importance of minimizing qubit requirements. While their theoretical analysis aligns with our logarithmic encoding findings, they did not implement and evaluate these encodings experimentally. Our work provides empirical

validation showing that theoretical qubit efficiency does not guarantee practical performance.

6.7.2 Comparison with Quantum Annealing Approaches

Irie et al. (2019)[16] formulated VRP as QUBO for quantum annealing, reporting better scalability (15–20 nodes) but lower solution quality compared to classical methods. The quantum annealing paradigm differs fundamentally from gate-based QAOA:

- **Annealing Advantages:** Natural constraint handling via QUBO penalties; potentially better scaling due to specialized hardware
- **QAOA Advantages:** More flexible circuit design; parameter optimization enables adaptation to problem structure

Our QAOA results (5–6 node scale) fall short of annealing’s reported scalability, suggesting annealing-based approaches may currently offer better near-term practicality for VRP. However, gate-based quantum computers offer broader algorithmic flexibility and clearer paths to fault-tolerant scaling.

6.7.3 Novel Contributions of This Work

Our research makes several contributions not found in prior literature:

First Systematic Encoding Comparison: We provide the first comprehensive experimental comparison of multiple qubit allocation strategies (naive adjacency, logarithmic permutation, node-visit-time) for quantum VRP, with detailed performance characterization.

Parameter Optimization Guidelines: Our identification of optimal QAOA depth ($p = 3–5$) and shot counts (512–1024) provides actionable guidance absent from previous studies, which often arbitrarily chose parameters.

Invalid Bitstring Analysis: We extensively characterize invalid bitstring types and rates, revealing fundamental constraints not adequately addressed in prior work. This analysis explains previously puzzling performance limitations.

Theory-Practice Gap Documentation: Our demonstration that logarithmic permutation encoding, despite theoretical optimality, proves impractical serves as a cautionary tale for quantum algorithm design, emphasizing the importance of considering practical implementation constraints.

6.8 Practical Recommendations for Quantum VRP Research

Based on our findings, we offer concrete recommendations for researchers working on quantum VRP optimization:

6.8.1 Encoding Strategy Selection

1. **Default to node-visit-time encoding** for gate-based quantum computers unless specific problem structure suggests alternatives
2. **Avoid naive adjacency encoding** for instances exceeding 4 nodes due to prohibitive failure rates
3. **Explore hybrid encodings** that combine node-visit-time’s structure with problem-specific innovations (e.g., pre-fixed depot positions, customer clusters)
4. **Do not pursue logarithmic permutation encoding** despite qubit efficiency due to insurmountable Hamiltonian locality challenges

6.8.2 QAOA Parameter Selection

1. **Start with $p = 3$** as default QAOA depth, testing $p = 2, 4, 5$ if computational budget allows
2. **Use 512–1024 shots** for measurement; avoid excessive shot counts that waste resources without improving quality
3. **Invest in parameter optimization** (improved classical optimizers, better initialization) rather than naively increasing depth or shots
4. **Consider adaptive depth selection** where p scales with problem size or adjusts based on convergence metrics

6.8.3 Hardware and Implementation

1. **Prioritize low-noise quantum processors** even if they have fewer qubits. In this case, gate fidelity matters more than raw qubit count for NISQ optimization
2. **Optimize circuit compilation** using problem-specific techniques (e.g., exploiting VRP symmetries, reducing SWAP overhead)
3. **Implement error mitigation** from the start rather than as an afterthought, zero-noise extrapolation provides meaningful quality improvements
4. **Validate with classical optimal solutions** for small instances before scaling to larger problems where verification becomes intractable

6.8.4 Research Directions

1. **Investigate hybrid quantum-classical decomposition** that leverages quantum optimization for tractable subproblems while using classical methods for global coordination
2. **Explore alternative quantum algorithms** beyond QAOA (e.g., variational quantum eigensolvers, quantum approximate counting, Grover-based approaches)
3. **Develop problem-specific mixer Hamiltonians** that preserve feasibility constraints, potentially reducing invalid bitstring rates
4. **Study warm-start initialization** from classically-generated feasible solutions rather than uniform superposition

7 Conclusion

This section concludes the thesis by synthesizing our findings on quantum VRP optimization. We summarize the key findings on encoding strategy performance, QAOA parameters, and NISQ limitations. We then outline the research contributions, acknowledge limitations and challenges, and propose future research directions. The section closes with remarks on broader impact and the outlook for quantum optimization in combinatorial problems.

7.1 Summary of Key Findings

Our research yields several significant conclusions:

7.1.1 Encoding Strategy Performance

Node-visit-time encoding emerges as the superior practical choice for quantum VRP solving on NISQ hardware, achieving a 91.3% success rate with 92.8% average optimality across 23 problem instances. This substantially outperforms naive adjacency matrix encoding’s 43.5% success rate despite the latter’s 98.7% optimality when successful. The performance advantage stems from node-visit-time encoding’s favorable constraint structure, better state space utilization, and improved error resilience.

Logarithmic permutation encoding, while theoretically optimal in qubit count ($\lceil \log_2(n!) \rceil$ qubits), proves impractical due to fundamental challenges in Hamiltonian construction and cost function evaluation. This finding illuminates a broader lesson: theoretical metrics like qubit count do not always predict practical performance. Algorithm designers must also consider circuit expressiveness, Hamiltonian locality, and implementation feasibility.

7.1.2 QAOA Parameter Optimization

Systematic parameter studies reveal that QAOA depth $p = 3\text{--}5$ and measurement shot counts of 512–1024 provide optimal performance on current NISQ hardware. Deeper circuits and excessive measurements yield diminishing returns due to the fundamental trade-off between quantum advantage and error accumulation. This finding enables researchers to achieve near-optimal results while minimizing quantum computation costs: a $4\times$ reduction in shots ($4096 \rightarrow 1024$) causes negligible performance degradation while substantially reducing execution time and cloud computing expenses.

7.1.3 NISQ Limitations and Scalability

Invalid bitstring rates exceeding 90% and scalability breakdown beyond 5–6 nodes establish realistic expectations for near-term quantum VRP optimization. These limitations stem from multiple factors: sparse valid solution subspaces, accumulated gate errors, limited circuit depth, and insufficient Hamiltonian expressiveness. Current NISQ devices can reliably solve VRP instances with up to 5 nodes, but larger problem scales remain beyond practical reach without significant hardware improvements or algorithmic innovations.

Performance scales gracefully for node-visit-time encoding through 5 nodes (optimality $> 92\%$, success rate $> 90\%$) but degrades sharply at 6 nodes. In contrast, naive adjacency encoding exhibits catastrophic failure beyond 4 nodes, confirming that encoding strategy critically impacts scalability thresholds.

7.1.4 Quantum Circuit Behavior

Despite high noise levels and invalid bitstring rates, QAOA circuits successfully concentrate probability mass around optimal solutions. Cost distribution analysis reveals trimodal distributions with strong peaks at optimal costs, demonstrating that the quantum optimization process effectively identifies high-quality solutions even when the majority of measurements yield invalid routes. This finding suggests that improved post-processing or amplitude amplification techniques could extract greater value from existing quantum circuits.

7.1.5 Classical vs. Quantum Performance

At current problem scales (3–6 nodes), classical methods retain clear advantages in solution quality (100% vs. 91.3% success rate), computational efficiency (2–45 seconds vs. 40 seconds), and reliability. However, this comparison should be contextualized within the early stage of quantum computing development. Classical VRP algorithms have benefited from six decades of refinement, while quantum approaches remain in their infancy with immature hardware and nascent algorithmic techniques.

7.2 Research Contributions

This thesis makes several significant contributions to quantum optimization research:

1. **First Systematic Encoding Comparison:** We provide the first comprehensive, experimental comparison of qubit allocation strategies for quantum VRP solving, encompassing theoretical analysis, implementation details, and extensive performance characterization across multiple metrics and problem instances.
2. **Practical Performance Validation:** Through experiments on real quantum hardware (IBM Quantum Cloud) rather than idealized simulators, we establish realistic performance baselines and identify practical challenges (invalid bitstrings, error accumulation, scalability limits) that theoretical studies often overlook.
3. **Actionable Parameter Guidelines:** Our identification of optimal QAOA parameters ($p = 3-5$, 512–1024 shots) provides concrete guidance for practitioners, enabling efficient resource utilization and improved solution quality.
4. **Theory-Practice Gap Analysis:** We rigorously demonstrate why logarithmic permutation encoding fails in practice despite theoretical optimality, contributing important insights for quantum algorithm design principles that extend beyond VRP to combinatorial optimization broadly.
5. **NISQ Characterization:** Our detailed analysis of invalid bitstring types, error impacts, and scalability thresholds establishes realistic expectations for near-term quantum optimization, helping the research community prioritize productive directions over unfruitful approaches.
6. **Real-World Case Study Integration:** By grounding our research in City of Casey waste collection data, we demonstrate how spatial datasets can be transformed into VRP formulations suitable for quantum optimization, providing a template for other practical applications.

7.3 Limitations and Challenges

Several limitations constrain our research and suggest caution in generalizing findings:

7.3.1 Hardware Specificity

All experiments utilized IBM Quantum Cloud infrastructure. Results may vary on alternative quantum platforms (Google Sycamore, IonQ trapped ions, Rigetti superconducting qubits) due to different error characteristics, connectivity topologies, and gate sets. However, the fundamental challenges identified (invalid bitstrings, error accumulation, scalability limits) likely persist across platforms given shared NISQ constraints.

7.3.2 Problem Instance Scope

We evaluated 23 VRP instances ranging from 3–6 nodes derived from a single real-world dataset (City of Casey waste collection). While this provides meaningful practical grounding, the limited problem diversity prevents strong claims about generalization to other VRP variants (time windows, multiple depots, heterogeneous vehicles, stochastic demands) or fundamentally different problem structures. Future work should validate findings across broader problem classes.

7.3.3 Classical Baseline Selection

Our classical benchmarks used exact branch-and-bound for small instances and standard heuristics for comparison. State-of-the-art industrial VRP solvers (incorporating cutting planes, column generation, adaptive large neighborhood search) would likely achieve even better performance, further widening the gap with current quantum methods. However, our goal was establishing quantum encoding comparisons rather than comprehensive quantum-classical competition.

7.3.4 Algorithm Scope

We focused exclusively on QAOA as the quantum optimization algorithm. Alternative approaches (quantum walks, variational quantum eigensolvers, quantum annealing, Grover-based methods) might yield different performance characteristics and potentially better results. QAOA represents just one point in the quantum algorithm design space, and our encoding insights may not fully transfer to other algorithmic frameworks.

7.3.5 Error Mitigation Absence

We did not implement advanced error mitigation techniques (zero-noise extrapolation, probabilistic error cancellation, Clifford data regression) that could potentially improve performance. Our results therefore represent lower bounds on achievable quality. Dedicated error mitigation efforts might substantially enhance success rates and solution quality, though at increased computational cost.

7.4 Future Research Directions

Our findings suggest several promising avenues for future investigation:

7.4.1 Algorithmic Innovations

Feasibility-Preserving QAOA: Design mixing Hamiltonians that maintain constraint satisfaction throughout quantum evolution, potentially reducing invalid bitstring rates

from 90%+ to manageable levels. This requires identifying problem-specific unitary operators that explore solution space while preserving validity.

Warm-Start Quantum Optimization: Initialize QAOA from classically-generated feasible solutions rather than uniform superposition. This hybrid approach could combine classical rapid feasibility generation with quantum refinement, potentially achieving better quality than either method alone.

Adaptive Circuit Depth: Develop heuristics for automatically selecting QAOA depth based on problem characteristics, convergence metrics, or circuit simulation. Adaptive approaches could optimize the quality-error trade-off on a per-instance basis rather than using fixed depths.

Quantum-Classical Decomposition: Explore hybrid methods that decompose large VRP instances into quantum-tractable subproblems (routing within customer clusters) coordinated by classical algorithms (connecting clusters). This divide-and-conquer strategy might enable scaling to industrial problem sizes (100–1000 customers).

7.4.2 Error Mitigation and Hardware Optimization

Comprehensive Error Mitigation: Implement and evaluate state-of-the-art error mitigation techniques including zero-noise extrapolation, probabilistic error cancellation, and symmetry verification. Quantify performance improvements versus computational overhead to establish cost-benefit profiles.

Circuit Compilation Optimization: Develop VRP-specific circuit compilation techniques that exploit problem structure (symmetries, constraints, locality) to minimize gate counts, reduce SWAP overhead, and improve overall circuit fidelity.

Alternative Quantum Platforms: Validate findings on diverse quantum hardware architectures (trapped ions, neutral atoms, photonic systems) to identify platform-specific advantages and establish platform-agnostic best practices.

7.4.3 Algorithmic Comparisons

Quantum Walk Implementations: Implement and benchmark quantum walk-based VRP solvers, comparing performance, scalability, and resource requirements against QAOA. Quantum walks offer alternative solution space exploration mechanisms that might overcome QAOA’s limitations.

Quantum Annealing vs. Gate-Based: Conduct direct head-to-head comparisons between quantum annealing (D-Wave systems) and gate-based QAOA (IBM, Google systems) on identical VRP instances to empirically establish relative strengths.

Hybrid Quantum-Classical Methods: Explore richer hybrid architectures where quantum and classical components interleave more tightly, such as quantum subroutines within classical metaheuristics or classical repair operators post-quantum generation.

7.4.4 Application Extensions

VRP Variant Exploration: Extend quantum approaches to VRP variants with time windows, capacity constraints, multiple depots, and dynamic requests. Each variant introduces unique challenges that may favor different encoding strategies or require novel algorithmic innovations.

Related Combinatorial Problems: Apply insights from quantum VRP solving to related problems (traveling salesman, bin packing, scheduling, graph coloring) to establish whether findings generalize across combinatorial optimization domains.

Real-World Deployment Studies: Collaborate with logistics companies to test quantum VRP optimization on industrial problem instances, evaluate integration challenges, and quantify practical business value of current quantum capabilities.

7.4.5 Theoretical Investigations

Approximation Guarantees: Develop theoretical approximation bounds for QAOA-based VRP solving as functions of circuit depth, problem size, and encoding strategy. Rigorous approximation guarantees would complement our empirical findings.

Quantum Advantage Thresholds: Analytically or empirically identify problem size thresholds where quantum algorithms transition from inferior to superior performance compared to classical methods, considering realistic hardware improvement trajectories.

Encoding Theory: Formalize principles for designing effective qubit encodings that balance qubit efficiency, Hamiltonian locality, constraint satisfaction, and error resilience. Such a theoretical framework could guide encoding design for new problems.

7.5 Broader Impact and Significance

Beyond specific technical contributions, this research advances quantum computing’s practical application to real-world optimization problems:

Realistic Expectations: By transparently reporting both successes and failures, we help establish realistic expectations for near-term quantum optimization. Overhyped claims of imminent quantum advantage undermine the field’s credibility; honest assessment of current capabilities and limitations enables informed decision-making by researchers, investors, and practitioners.

Best Practices: Our practical recommendations (encoding selection, parameter choices, implementation strategies) provide actionable guidance that accelerates future research by helping newcomers avoid unproductive approaches and focus efforts on promising directions.

Interdisciplinary Bridge: This work connects quantum computing research with operations research and logistics optimization communities. By demonstrating quantum algorithms on real-world VRP instances, we facilitate dialogue between communities that often remain siloed, potentially inspiring cross-pollination of ideas.

Educational Resource: The comprehensive treatment of encoding strategies, implementation details, and experimental methodology serves as an educational resource for students and researchers entering quantum optimization, providing a detailed roadmap from theory to implementation.

7.6 Closing Remarks

The journey toward practical quantum advantage for combinatorial optimization remains in its early stages. Current NISQ devices cannot yet outperform classical methods for Vehicle Routing Problems at practical scales, and significant challenges persist: high invalid bitstring rates, severe scalability limitations, and noise-induced errors substantially constrain performance.

However, our research demonstrates that even within these constraints, careful algorithm design matters profoundly. Node-visit-time encoding’s $2\times$ success rate advantage over naive adjacency encoding shows that encoding strategy choices critically impact practical performance. The identification of optimal QAOA parameters reveals that informed tuning can maximize the value extracted from limited quantum resources. These findings suggest that as quantum hardware improves, and improvements are occurring rapidly, algorithmic innovations will be equally crucial for realizing quantum optimization’s potential.

The path forward requires sustained effort across multiple fronts: quantum hardware engineers reducing error rates and increasing qubit counts, quantum algorithm researchers developing more sophisticated optimization techniques, classical-quantum hybrid method designers finding optimal integration strategies, and domain experts identifying application areas where quantum advantages might first emerge.

We remain cautiously optimistic about quantum optimization’s future. While current results fall short of classical performance, the fundamental principles, quantum superposition enabling parallel exploration, interference amplifying high-quality solutions, entanglement correlating variables, remain sound. With continued advancement in both

hardware and algorithms, quantum optimization may eventually fulfill its promise of revolutionizing how we solve computationally hard problems. This thesis contributes one step along that long and challenging journey.

A Software Implementation and Repository

The complete implementation of this research is publicly available in a GitHub repository for reproducibility and further development. This appendix describes the repository structure and provides guidance for accessing the experimental code and data.

A.1 Repository Access

The main repository is available at:

- **Main GitHub Repository:** <https://github.com/drestanto/quantumvrp>
- **Files Overview:** <https://github.com/drestanto/quantumvrp/blob/main/README.md>
- **Code and Step-by-Step Guide:** <https://github.com/drestanto/quantumvrp/tree/main/code>

A.2 Repository Structure

The repository is organized into the following directories:

Directory	Purpose
<code>docs/</code>	Foundational materials, literature analysis, theoretical frameworks, and project documentation
<code>code/</code>	Implementation files for problem generation, quantum solvers, hybrid approaches, and execution scripts
<code>gathered_data/</code>	Curated datasets, experimental logs, and primary findings
<code>raw_data/</code>	Unprocessed source materials, reference benchmarks, and computational outputs
<code>manuscripts/</code>	Academic papers, technical reports, and presentation materials

A.3 Running the Experiments

To reproduce the experiments presented in this thesis:

1. Clone the repository:

```
git clone https://github.com/drestanto/quantumvrp.git
cd quantumvrp
```

2. Navigate to the `code/` directory for implementation files
3. Follow the step-by-step instructions in the `code/README.md` file

4. The experimental data used in this thesis can be found in `gathered.data/`

Each subdirectory contains its own README file with specific instructions for running particular experiments. The implementation uses Python with Qiskit for quantum circuit construction and IBM Quantum Cloud for experimental execution.

References

- [1] M. Yousefikhoshbakht and E. Khorram. Solving the vehicle routing problem by a hybrid meta-heuristic algorithm. *Journal of Industrial Engineering International*, 8:1–9, 2012.
- [2] K. Braekers, K. Ramaekers, and I. Van Nieuwenhuysse. The vehicle routing problem: State of the art classification and review. *Computers & Industrial Engineering*, 99:300–313, 2016.
- [3] City of Casey. Waste facility locations. <https://data.casey.vic.gov.au/explore/dataset/waste-facility-locations>, n.d.
- [4] City of Casey. Public litter bins. <https://data.casey.vic.gov.au/explore/dataset/public-litter-bins>, n.d.
- [5] City of Casey. Waste collection areas. <https://data.casey.vic.gov.au/explore/dataset/waste-collection-area>, n.d.
- [6] VicRoads. Victoria road network. <https://vicdata.vicroads.vic.gov.au/portal/home/group.html?id=82b544768d5b4c3cbd4a79a4df322984>, n.d.
- [7] VicRoads Open Data. Traffic volume. <https://vicroadsopendata-vicroadsmaps.opendata.arcgis.com/datasets/vicroadsmaps::traffic-volume/about>, n.d.
- [8] City of Casey. Rainfall data. <https://data.casey.vic.gov.au/explore/dataset/rainfall-data>, n.d.
- [9] Data VIC. Victoria road crash data. <https://discover.data.vic.gov.au/dataset/victoria-road-crash-data>, n.d.
- [10] E. Farhi, J. Goldstone, and S. Gutmann. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028*, 2014.
- [11] S. Marsh and J. B. Wang. A quantum walk-assisted approximate algorithm for bounded np optimisation problems. *Quantum Information Processing*, 18(3):61, 2019.
- [12] P. Toth and D. Vigo. *Vehicle Routing: Problems, Methods, and Applications*. SIAM, Philadelphia, PA, USA, 2014.
- [13] G. Laporte. Fifty years of vehicle routing. *Transportation Science*, 43(4):408–416, 2009.
- [14] T. Bennett, E. Matwiejew, S. Marsh, and J. B. Wang. Quantum walk-based vehicle routing optimisation. *Frontiers in Physics*, 9:730856, 2021.

- [15] M. Alsaiyari and M. Felemban. Variational quantum algorithms for solving vehicle routing problem. In *Proc. 2023 Int. Conf. Smart Computing Applications (ICSCA)*, pages 1–4, 2023.
- [16] H. Irie, G. Wongpaisarnsin, M. Terabe, A. Miki, and S. Taguchi. Quantum annealing of vehicle routing problem with time, state and capacity. In *Quantum Technology and Optimization Problems: First International Workshop, QTOP 2019*, pages 145–156. Springer, 2019.
- [17] D. Fitzek, T. Ghandriz, L. Laine, M. Granath, and A. F. Kockum. Applying quantum approximate optimization to the heterogeneous vehicle routing problem. *Scientific Reports*, 14(1):25415, 2024.
- [18] I. D. Leonidas, A. Dukakis, B. Tan, and D. G. Angelakis. Qubit efficient quantum algorithms for the vehicle routing problem on nisq processors. *arXiv preprint arXiv:2306.08507*, 2023.
- [19] S. Marsh and J. B. Wang. Combinatorial optimization via highly efficient quantum walks. *Physical Review Research*, 2(2):023302, 2020.
- [20] Toufan Diansyah Tambunan, Andriyan Bayu Suksmono, Ian Joseph Matheus Edward, and Rahmat Mulyawan. Quantum annealing for vehicle routing problem with weighted segment. In *AIP Conference Proceedings*, volume 2906. AIP Publishing, November 2023.
- [21] N. Mohanty, B. K. Behera, and C. Ferrie. Solving the vehicle routing problem via quantum support vector machines. *Quantum Machine Intelligence*, 6(1):34, 2024.
- [22] Radhakrishnan Balu and Ahmed Younes. Solving the travelling salesman problem using a single qubit. *arXiv preprint arXiv:2407.17207*, 2024.
- [23] Sungjin Baek, Zhihao Qian, Yihan Zhang, and Frederic T Chong. Type-based qubit allocation for a first-order quantum programming language. *arXiv preprint arXiv:2306.01856*, 2023.
- [24] Yingkai Shi, Yudong Cao, Hanrui Tang, Huanrui Fan, Ke Pei, Tianshi Wu, and Jason Cong. Qubit allocation for distributed quantum computing. *2023 IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 695–709, 2023.
- [25] Guoming Li, Yu Ding, Yuan Xie, and Mingyu Zhou. An exact qubit allocation approach for nisq architectures. *Quantum Information Processing*, 19(2):1–19, 2020.
- [26] Bowen Wang, Ziqi Li, Ping Chen, Wenjun Hu, and Yuanyuan Ma. Quantum annealing and graph neural networks for solving tsp with qubo. *arXiv preprint arXiv:2402.14036*, 2024.

- [27] Gary Kochenberger, Jin-Kao Hao, Fred Glover, Matthew Lewis, Haibo Wang, Yang Wang, and Taylor Wuster. Ising formulations of many np problems. *Frontiers in Physics*, 2:5, 2014.
- [28] Zhen Yu, Shuai Sun, Hongjun Xu, and Jianjun Wu. A hybrid solution method for the capacitated vehicle routing problem using a quantum annealer. *arXiv preprint arXiv:1811.07403*, 2018.
- [29] Gautham Surya, Moin Khan, and Andre Chailloux. An efficient quantum algorithm for the traveling salesman problem. *Cryptology ePrint Archive*, 2024:626, 2024.
- [30] Sitao Liu and Yongshan Lu. Optimization driven quantum circuit reduction. *arXiv preprint arXiv:2502.14715*, 2024.
- [31] Andrew Lucas. Ising formulations of many NP problems. *arXiv preprint arXiv:1302.5843*, 2013. *Frontiers in Physics* 2:5 (2014).
- [32] Taisuke Izumi and François Le Gall. Quantum distributed algorithm for the all-pairs shortest path problem in the congest-clique model. In *Proceedings of the 38th ACM Symposium on Principles of Distributed Computing (PODC)*, pages 84–93, 2019. Accepted to PODC’19.
- [33] Susanna Caroppo, Giordano Da Lozzo, Giuseppe Di Battista, Michael T. Goodrich, and Martin Nöllenburg. Quantum speedups for polynomial-time dynamic programming algorithms. *arXiv preprint arXiv:2507.00823*, 2025. Submitted 1 July 2025.
- [34] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028*, 2014.
- [35] Yuichiro Minato. Two-step qaoa: Enhancing quantum optimization by decomposing one-hot constraints in qubo formulations. *arXiv preprint arXiv:2408.05383*, 2025. Version v2, submitted 23 February 2025.
- [36] Kapil Goswami, Gagan Anekonda Veereshi, Peter Schmelcher, and Rick Mukherjee. Solving the travelling salesman problem using a single qubit. *arXiv preprint arXiv:2407.17207*, 2024. Version v2, submitted 24 July 2024, revised 23 December 2024.
- [37] Taisuke Izumi and François Le Gall. Quantum distributed algorithm for the all-pairs shortest path problem in the congest-clique model. In *Proceedings of the 38th ACM Symposium on Principles of Distributed Computing (PODC)*, pages 84–93, 2019.
- [38] R. Duan, J. Mao, X. Mao, X. Shu, and L. Yin. Breaking the sorting barrier for directed single-source shortest paths. *arXiv:2504.17033 [cs.DS]*, 2025.